

RTL Week 2 Labs

Date: 28 June 2026

Submitted To: Sir Khalil

Submitted By: Jaleed Abdur Rehman Khan

GitHub Repository Link: https://github.com/JaleedAbdurRehmanKhan/RTL_week_2

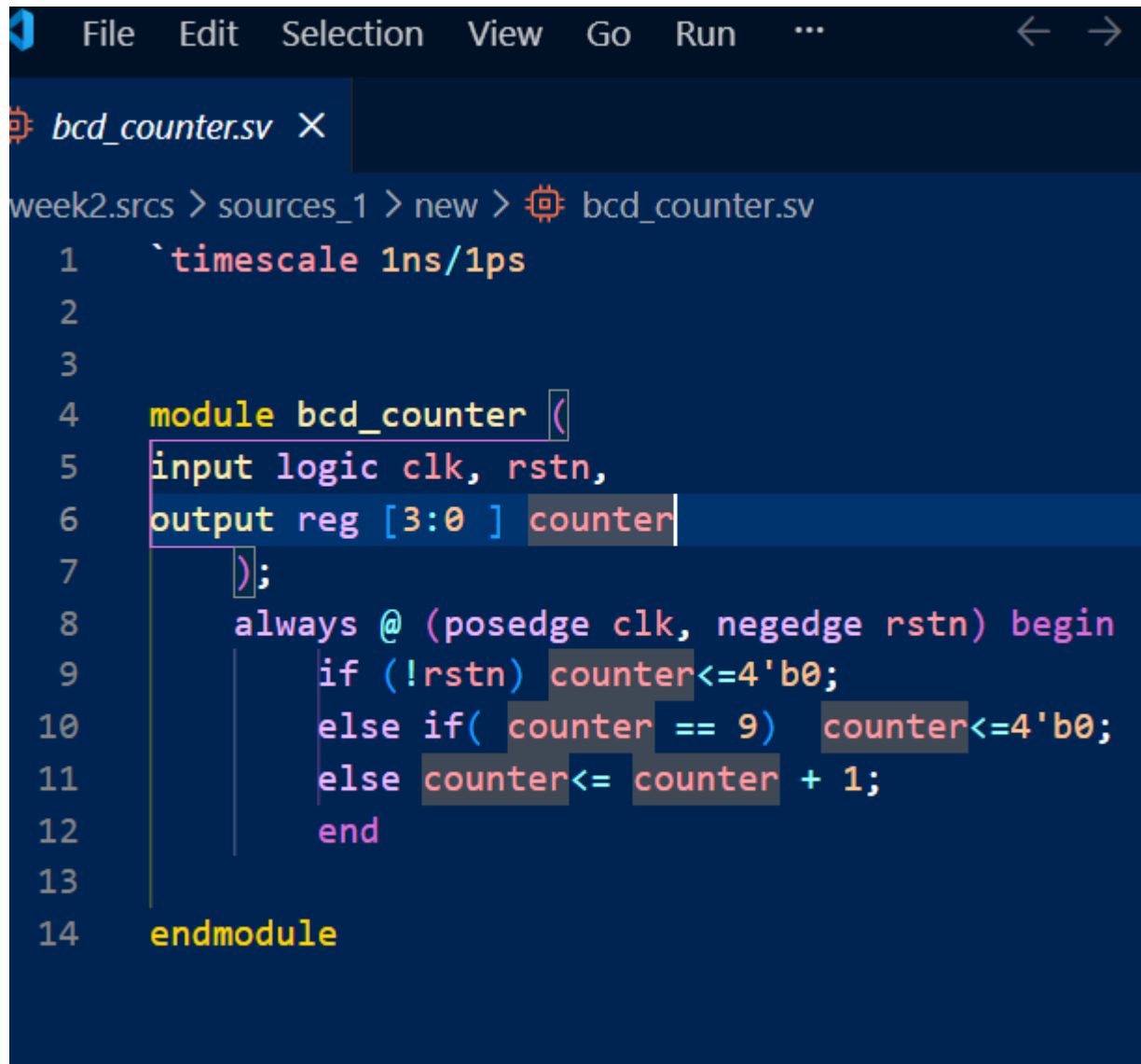
Contents

Task 1: BCD Counter	4
Design Code	4
Waveform	6
Schematic.....	6
Task 2: Asynchronous Up/Down Counter	7
Design Code	7
Testbench Code	8
Task 3: Synchronous Up/Down Counter	10
Design Code	10
Testbench Code	10
Task 4: Mod-N Counter	13
Design Code	13
Testbench Code	14
Waveform	15
Schematic.....	15
Task 5: Frequency Divider.....	16
Design Code	16
Waveform	18
Schematic.....	18
Task 6: T Flip-Flop	19
Design Code	19
Testbench Code	20
Waveform	21
Schematic.....	21
Task 7: D Flip-Flop	22
Design Code	22
Testbench Code	23
Schematic.....	24
Task 8: JK Flip-Flop.....	25
Design Code	25
Testbench Code	26
Waveform	27
Schematic.....	27
Task 9: SR Flip-Flop.....	28
Design Code	28
Testbench Code	29
Waveform	30
Schematic.....	30

Task 10: T Flip-Flop Using D Flip-Flop	31
Design Code	31
Testbench Code	32
Task 11: SISO – Serial-In Serial-Out Shift Register	34
Design Code	34
Testbench Code	35
Waveform	36
Schematic	36
Task 12: SIPO – Serial-In Parallel-Out Shift Register	37
Design Code	37
Testbench Code	38
Schematic	38
Task 13: PIPO – Parallel-In Parallel-Out Shift Register	39
Design Code	39
Waveform	40
Task 14: PISO – Parallel-In Serial-Out Shift Register	42
Design Code	42
Testbench Code	43
Waveform	44
Schematic	44
Task 15: Bidirectional Shift Register	45
Design Code	45
Testbench Code	46
Waveform	47
Schematic	47
Task 16: Universal Shift Register	48
Design Code	48
Testbench Code	49
Waveform	50
Task 17: Pipelined ALU	52
Design Code	52
Testbench Code	54
Waveform	55
Schematic	55
Task 18: Pipelined Multiplier	56
Design Code	56
Testbench Code	57
Waveform	58

Task 1: BCD Counter

Design Code



```
File Edit Selection View Go Run ...  
bcd_counter.sv X  
week2.srscs > sources_1 > new > bcd_counter.sv  
1  `timescale 1ns/1ps  
2  
3  
4  module bcd_counter (  
5      input logic clk, rstn,  
6      output reg [3:0] counter  
7  );  
8      always @ (posedge clk, negedge rstn) begin  
9          if (!rstn) counter<=4'b0;  
10         else if( counter == 9) counter<=4'b0;  
11         else counter<= counter + 1;  
12     end  
13  
14 endmodule
```

Testbench

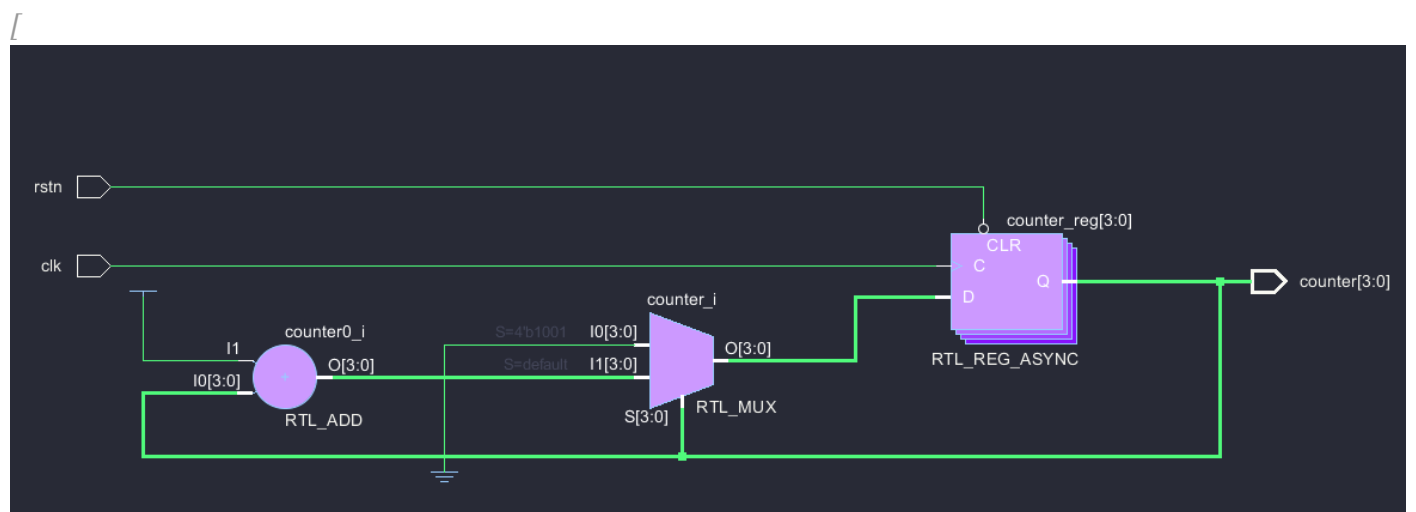
Code

```
1  `timescale 1ns / 1ps
2
3  module tb_bcd_counter(
4
5  );
6      // Signal declarations
7      logic clk;
8      logic rstn;
9      logic [3:0] q;
10
11      // Instantiate the Unit Under Test (UUT)
12      bcd_counter uut (
13          .clk(clk),
14          .rstn(rstn),
15          .counter(q)
16      );
17
18      // Clock generation (10ns period)
19      always #5 clk = ~clk;
20
21      // Test sequence
22      initial begin
23          // Initialize signals
24          clk = 0;
25          rstn = 0;
26          #10;
27          rstn=1;
28          #200;
29          $finish;
30      end
31  endmodule
32
```

Waveform



Schematic



Task 2: Asynchronous Up/Down Counter

Design Code

```
JaleedUSTP > RTL > RTLLab > week2 > week2.srsc > sources_1 > new >  async_up_down_counter.vh
1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////
3  // Company:
4
5
6  module async_up_down_counter(
7      input logic clk,
8      input logic rstn, up_down,
9      output reg [3:0] q
10 );
11
12
13     wire clk1 = (up_down) ? ~q[0] : q[0];
14     wire clk2 = (up_down) ? ~q[1] : q[1];
15     wire clk3 = (up_down) ? ~q[2] : q[2];
16
17     always @(posedge clk, negedge rstn) begin
18         if(!rstn) q[0] <= 1'b0;
19         else q[0] <= ~q[0];
20     end
21
22     always @(posedge clk1, negedge rstn) begin
23         if(!rstn) q[1] <= 1'b0;
24         else q[1] <= ~q[1]; // will count upwards if up_down 1
25     end
26
27     always @(posedge clk2, negedge rstn) begin
28         if(!rstn) q[2] <= 1'b0;
29         else q[2] <= ~q[2]; // will count upwards if up_down 1
30     end
31
32     always @(posedge clk3, negedge rstn) begin
33         if(!rstn) q[3] <= 1'b0;
34         else q[3] <= ~q[3]; // will count upwards if up_down 1
35     end
36
37
38
39 endmodule
```

Testbench Code

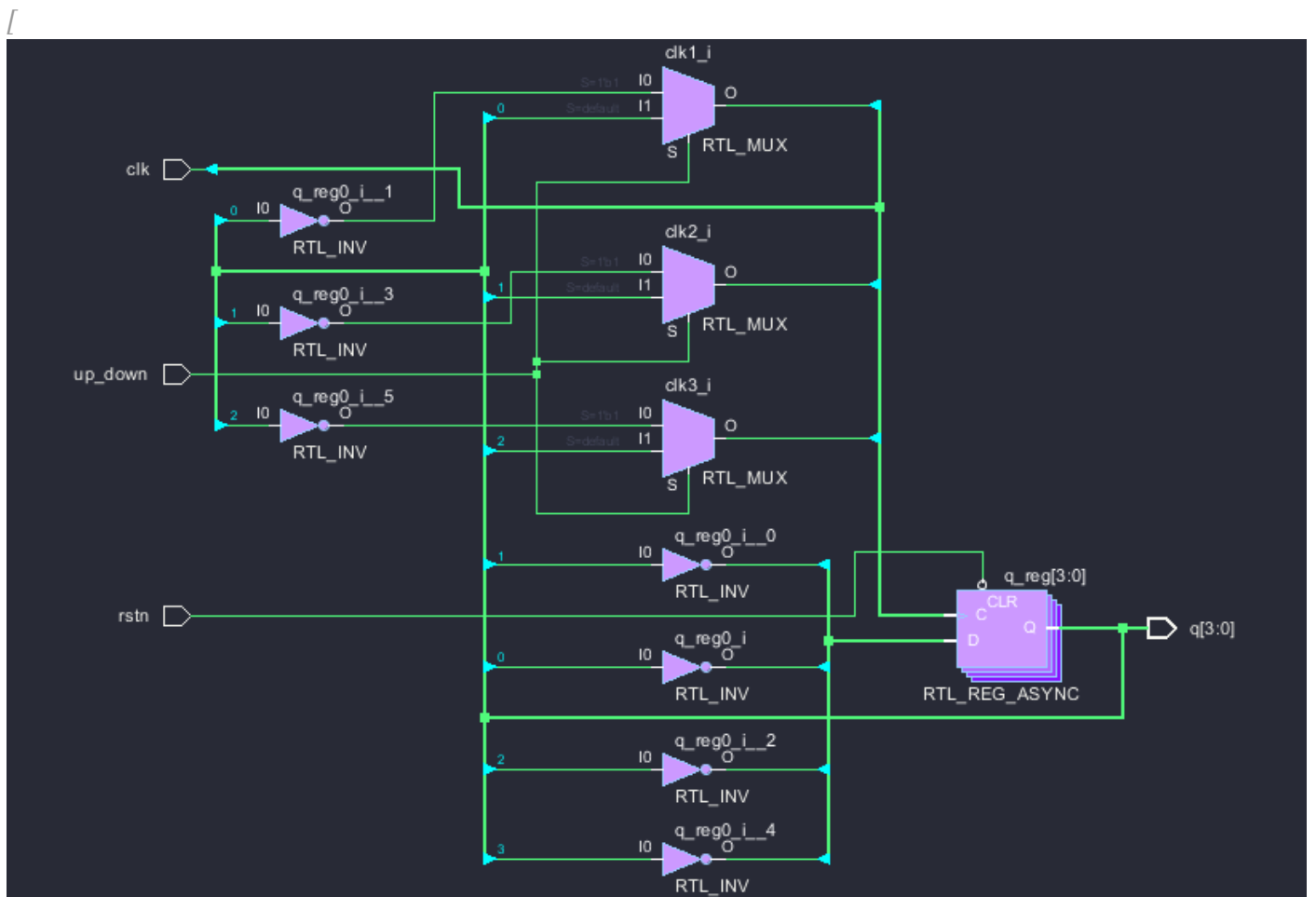
```
JaleedUSTP > RTL > RTLLab > week2 > week2.srcs > sim_1 > new > tb_async_up_down_counter.sv

1  `timescale 1ns / 1ps
2  module tb_async_up_down_counter();
3      // Signal declarations
4      logic clk;
5      logic rstn;
6      logic up_down;
7      logic [3:0] q;
8      // Instantiate the Unit Under Test (UUT)
9      async_up_down_counter uut (
10         .clk(clk),
11         .rstn(rstn),
12         .up_down(up_down),
13         .q(q)
14     );
15     // Clock generation (10ns period)
16     always #5 clk = ~clk;
17     // Test sequence
18     initial begin
19         // Initialize signals
20         clk = 0;
21         rstn = 0;
22         up_down = 0;
23         // Apply reset
24         #20 rstn = 1;
25         // Test Count Up (up_down = 1)
26         up_down = 1;
27         #100;
28
29         // Test Count Down (up_down = 0)
30         up_down = 0;
31         #100;
32
33         // Assert reset again
34         #10 rstn = 0;
35         #20 rstn = 1;
36
37         #50 $finish;
38     end
39
40 endmodule
```

Waveform



Schematic



Task 3: Synchronous Up/Down Counter

Design Code

```
JaleedUSTP > RTL > RTLLab > week2 > week2.srcs > sources_1 > new > sync_up_down_counter sv
1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
3  // Company:
4  //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
5
6
7  module sync_up_down_counter(
8      input clk,
9      input rstn,up_down,
10     output reg [3:0] q
11 );
12     always @ (posedge clk, negedge rstn) begin
13         if (!rstn) q<=4'b0;
14         else if(up_down) q <= q+1;
15         else q <= q-1;
16     end
17 end
18
19 endmodule
20
```

Testbench Code

JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sim_1 > new > tb_sync_up_down_counter.sv

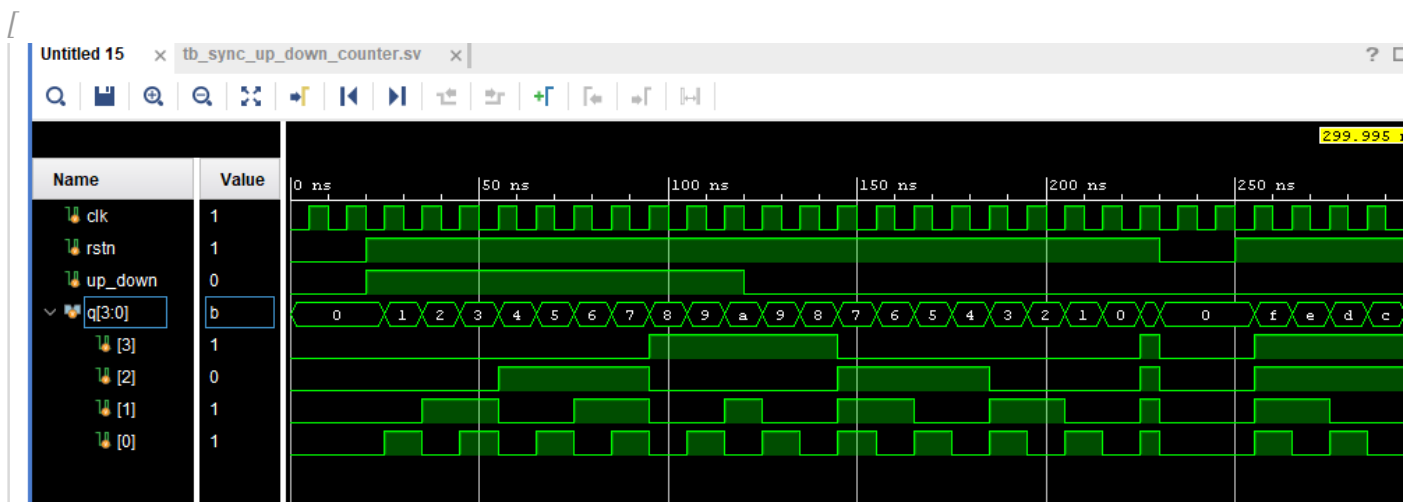
```
1  `timescale 1ns / 1ps
2
3  module tb_sync_up_down_counter();
4
5      // Signal declarations
6      logic clk;
7      logic rstn;
8      logic up_down;
9      logic [3:0] q;
10
11     // Instantiate the Unit Under Test (UUT)
12     sync_up_down_counter uut (
13         .clk(clk),
14         .rstn(rstn),
15         .up_down(up_down),
16         .q(q)
17     );
18
19     // Clock generation (10ns period)
20     always #5 clk = ~clk;
21
22     // Test sequence
23     initial begin
24         // Initialize signals
25         clk = 0;
26         rstn = 0;
27         up_down = 0;
28
29         // Apply reset
30         #20 rstn = 1;
31
32         // Test Count Up (up_down = 1)
33         up_down = 1;
34         #100;
35
36         // Test Count Down (up_down = 0)
37         up_down = 0;
38         #100;
39     end
```

```

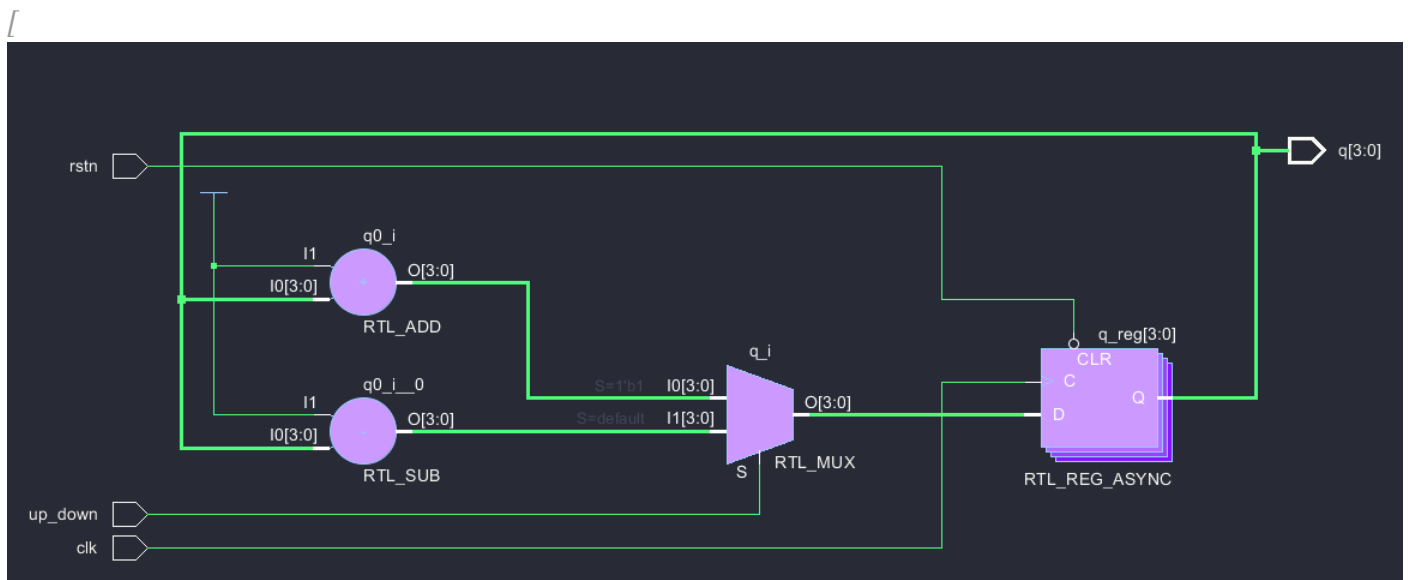
1      #10 rstn = 0;
2      #20 rstn = 1;
3
4      #50 $finish;
5  end
6
7  // Monitor changes
8  initial begin
9      $monitor("Time=%0t | rstn=%b | up_down=%b | q=%b", $time, rstn, up_down, q);
10 end
11
12 endmodule

```

Waveform



Schematic



Task 4: Mod-N Counter

Design Code

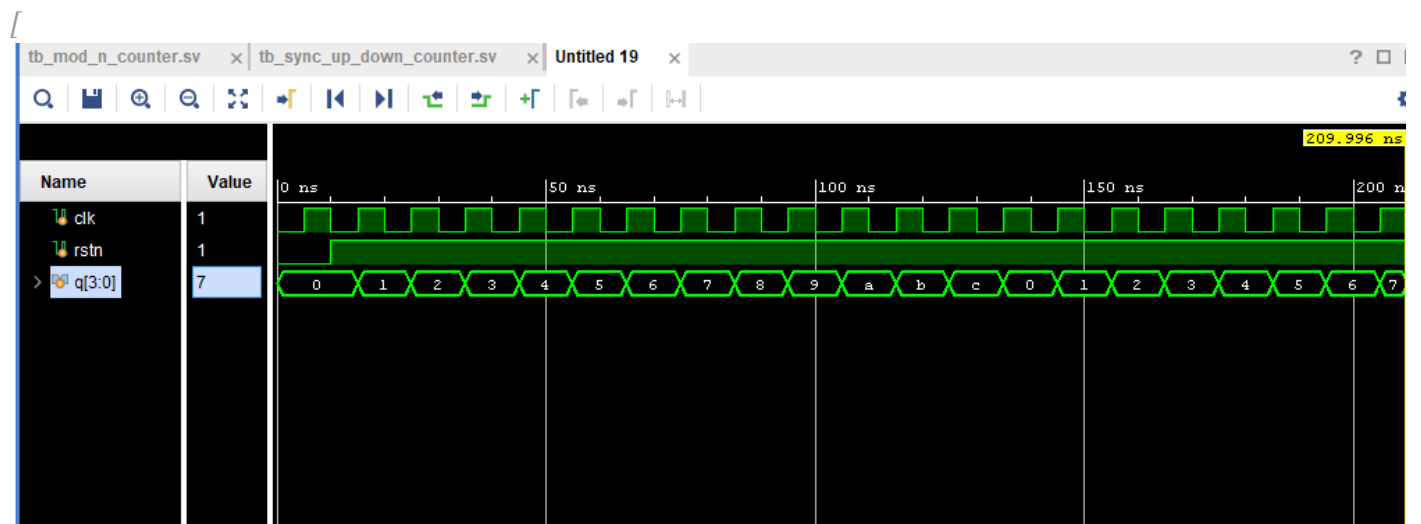
```
[
  tb_bcd_counter.sv  tb_siso.sv  tb_t_ff.sv  mod_n_counter.sv
JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sources_1 > new > mod_n_counter.sv
1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
3
4  module mod_n_counter #(parameter n=4)(
5  input logic clk, rstn,
6  output reg [3:0] counter
7  );
8  always @ (posedge clk, negedge rstn) begin
9      if (!rstn) counter<=4'b0;
10     else if( counter == n) counter<=4'b0;
11     else counter<= counter + 1;
12     end
13
14 endmodule
15
```

Testbench Code

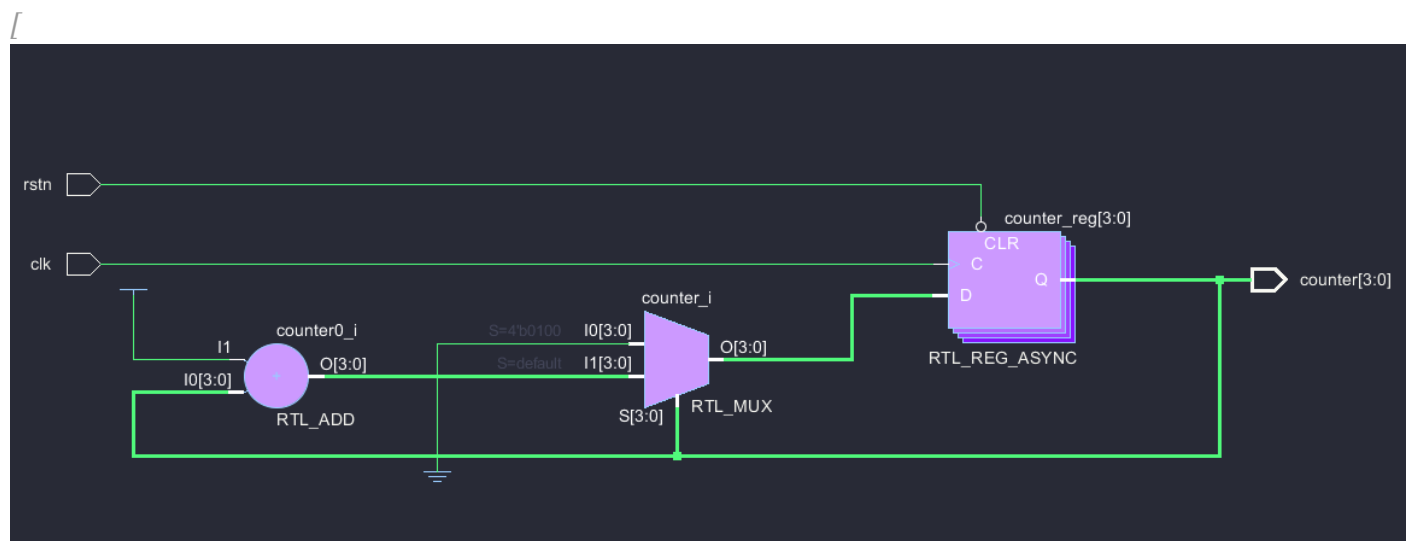
```
[
JaleedUSTP > RTL > RTLLab > week2 > week2.srcs > sim_1 > new > tb_mod_n_counter.sv

1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
3  //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
4
5
6  module tb_mod_n_counter(
7
8  );
9      // Signal declarations
10     logic clk;
11     logic rstn;
12     logic [3:0] q;
13
14     // Instantiate the Unit Under Test (UUT)
15     mod_n_counter #(.n(12)) uut (
16         .clk(clk),
17         .rstn(rstn),
18         .counter(q)
19     );
20
21     // Clock generation (10ns period)
22     always #5 clk = ~clk;
23
24     // Test sequence
25     initial begin
26         // Initialize signals
27         clk = 0;
28         rstn = 0;
29         #10;
30         rstn=1;
31         #50;
32         $finish;
33     end
34 endmodule
35
```

Waveform



Schematic



Task 5: Frequency Divider

Design Code

```
tb_bcd_counter.sv  tb_siso.sv  tb_t_ff.sv  freq_div.sv X
JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sources_1 > new > freq_div.sv
1  `timescale 1ns/1ps
2
3  module freq_div (
4      input clk_100_mhz, rstn,
5      output logic clk_25_mhz
6  );
7
8  logic counter;
9
10 always_ff @(posedge clk_100_mhz, negedge rstn) begin
11     if(!rstn) begin clk_25_mhz <= 0; counter <= 0; end
12     else if (counter == 1) begin
13         clk_25_mhz <= ~clk_25_mhz;
14         counter <= 0;
15     end
16     else begin
17         counter <= counter + 1;
18         clk_25_mhz <= clk_25_mhz;
19     end
20
21 end
22
23
24 endmodule
```

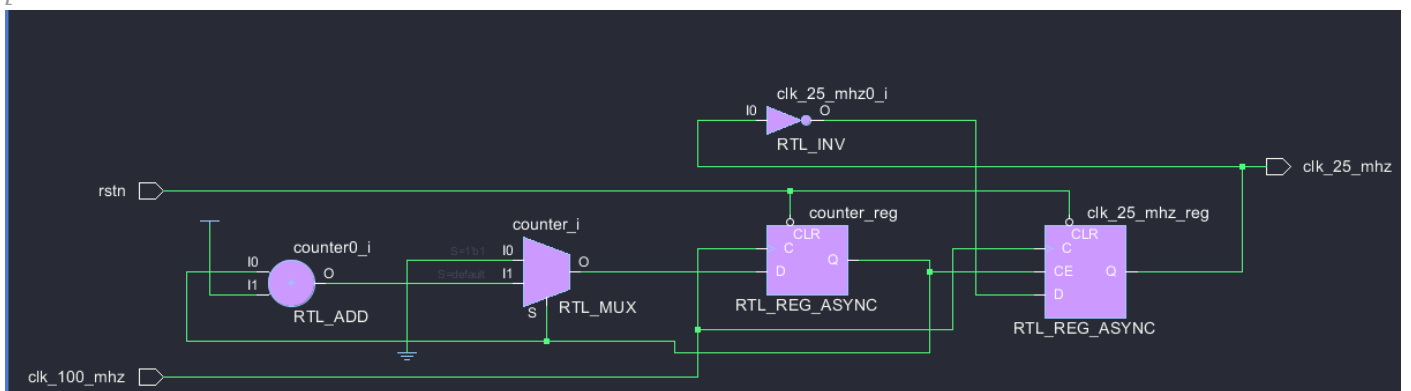
Testbench Code

tb_bcd_counter.sv tb_siso.sv tb_t_ff.sv tb_freq_div.sv X

JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sim_1 > new > tb_freq_div.sv

```
1  `timescale 1ns / 1ps
2
3  module tb_freq_div;
4
5  // freq_div Parameters
6  parameter PERIOD = 10;
7
8
9  // freq_div Inputs
10 reg clk_100_mhz = 0;
11 reg rstn = 0;
12
13 // freq_div Outputs
14 wire logic clk_25_mhz;
15
16
17 initial
18 begin
19     forever #(PERIOD) clk_100_mhz=~clk_100_mhz;
20 end
21
22 initial
23 begin
24     #(PERIOD*2) rstn = 1;
25 end
26
27 freq_div u_freq_div (
28     .clk_100_mhz ( clk_100_mhz ),
29     .rstn ( rstn ),
30
31     . clk_25_mhz ( clk_25_mhz )
32 );
33
34 initial
35 begin
36     #1000;
37     $finish;
38 end
39
40 endmodule
```

A



Task 6: T Flip-Flop

Design Code

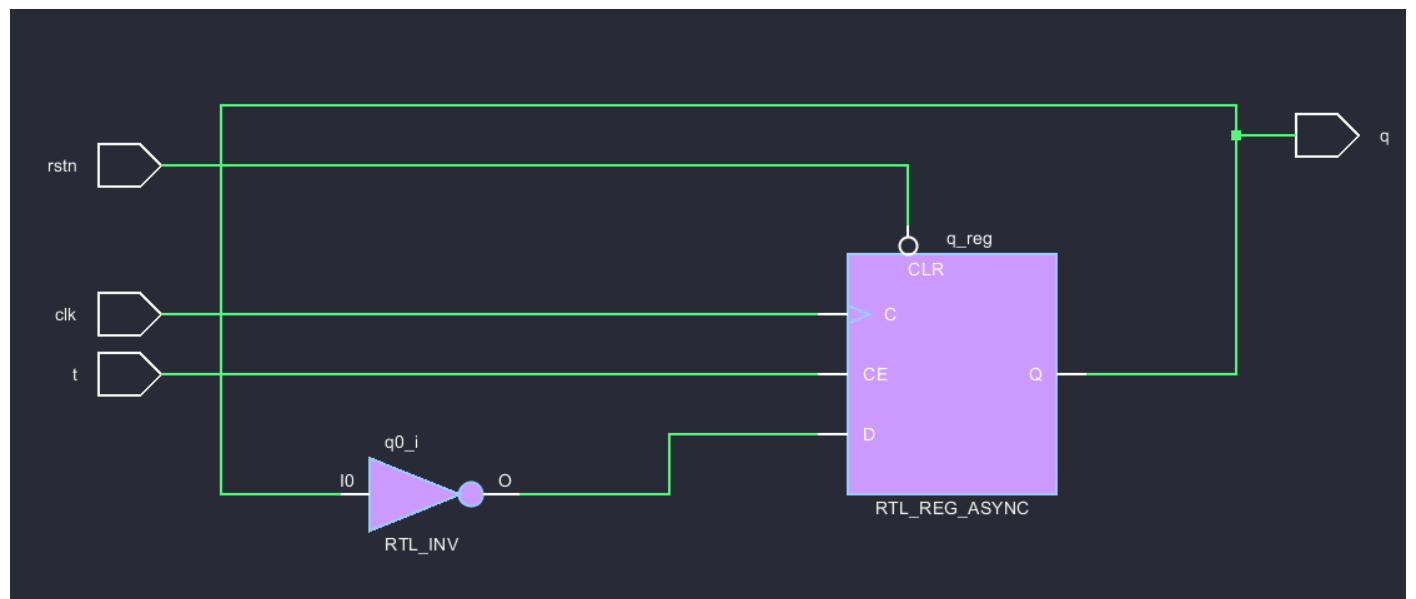
```
tb_bcd_counter.sv  tb_siso.sv  tb_t_ff.sv  t_ff.sv  X
aleedUSTP > RTL > RTLLab > week2 > week2.srcs > sources_1 > new > t_ff.sv
1  `timescale 1ns/1ps
2
3  module t_ff (
4      input rstn, clk, t,
5      output reg q
6  );
7
8  always @(posedge clk, negedge rstn) begin
9      if(!rstn) q<=0;
10     else if (t) q<=~q;
11     else q<=q;
12 end
13
14 endmodule
```

Testbench Code

```
tb_bcd_counter.sv  tb_siso.sv  tb_t_ff.sv  ●  tb_t_ff.sv
JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sim_1 > new > tb_t_ff.sv

1  `timescale 1ns/1ps
2
3  module tb_t_ff;
4
5  // t_ff Inputs
6  reg  rstn                = 0 ;
7  reg  clk                 = 0 ;
8  reg  t                   = 0 ;
9  // t_ff Outputs
10 wire q                  ;
11 initial
12 begin
13     forever #5  clk=~clk;
14 end
15 initial
16 begin
17     #10 rstn = 1;
18 end
19
20 t_ff u_t_ff (
21     .rstn        ( rstn ),
22     .clk         ( clk  ),
23     .t           ( t    ),
24
25     .q           ( q    )
26 );
27
28 initial
29 begin
30     t=1;
31     #10;
32     t=0;
33     #10;
34     t=1;
35     #100;
36
37     $finish;
38 end
39
40 endmodule
```

A



Task 7: D Flip-Flop

Design Code

```
tb_bcd_counter.sv  tb_siso.sv  d_ff.sv  X
JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sources_1 > new > d_ff.sv
1  `timescale 1ns / 1ps
2
3  module d_ff(
4  input logic clk,
5      input logic rstn,
6  input logic d,
7      output logic q
8  );
9
10     always_ff @(posedge clk, negedge rstn) begin
11         if(!rstn) q<=1'b0;
12         else q <=d;
13     end
14 endmodule
15
```

Testbench Code

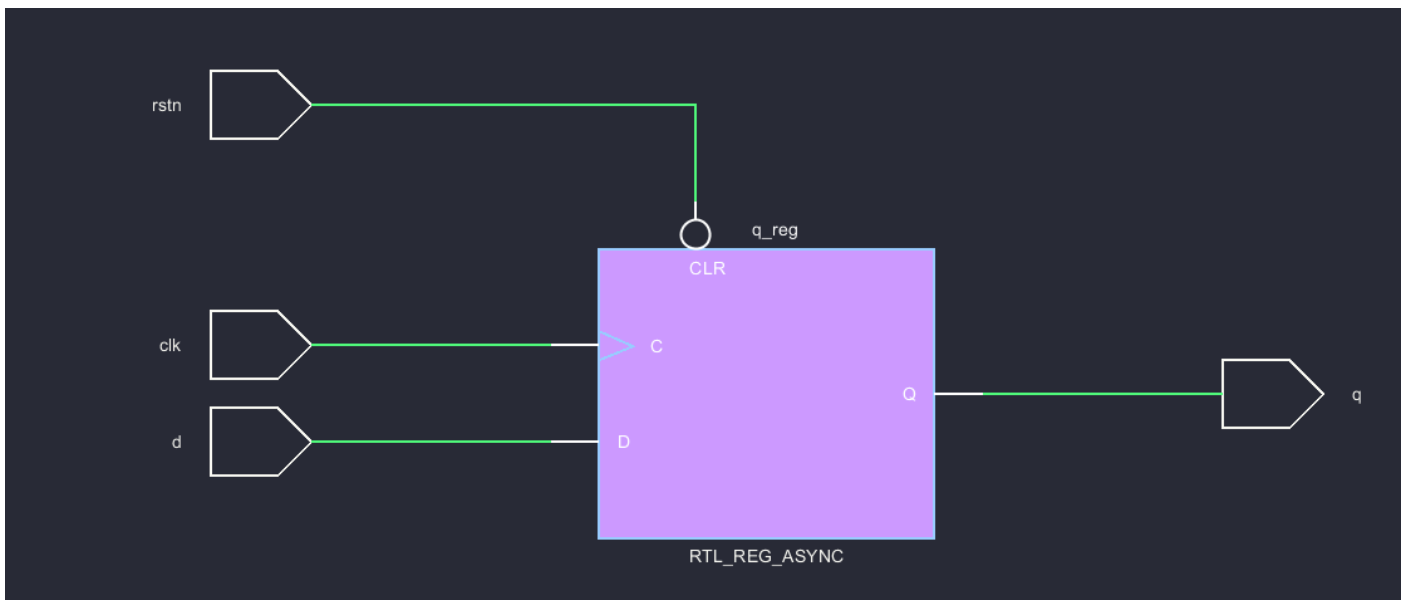
```
[
tb_bcd_counter.sv  tb_siso.sv  tb_d_ff.sv  X
maleedUSTP > RTL > RTLLab > week2 > week2.srsc > sim_1 > new > tb_d_ff.sv

1  `timescale 1ns / 1ps
2
3  module tb_d_ff();
4      logic clk;
5      logic rstn;
6      logic d;
7      logic q;
8
9      d_ff uut(
10         .clk(clk),
11         .rstn(rstn),
12         .d(d),
13         .q(q)
14     );
15
16     always #5 clk=~clk;
17
18     initial begin
19         clk=0;
20         rstn=0;
21         d=0;
22         #15 rstn=1;
23
24         d=1;
25         #10 d=0;
26         #10 d=1;
27         #10 d=1;
28         #10 $finish;
29     end
30 endmodule
31
```

Waveform



Schematic



Task 8: JK Flip-Flop

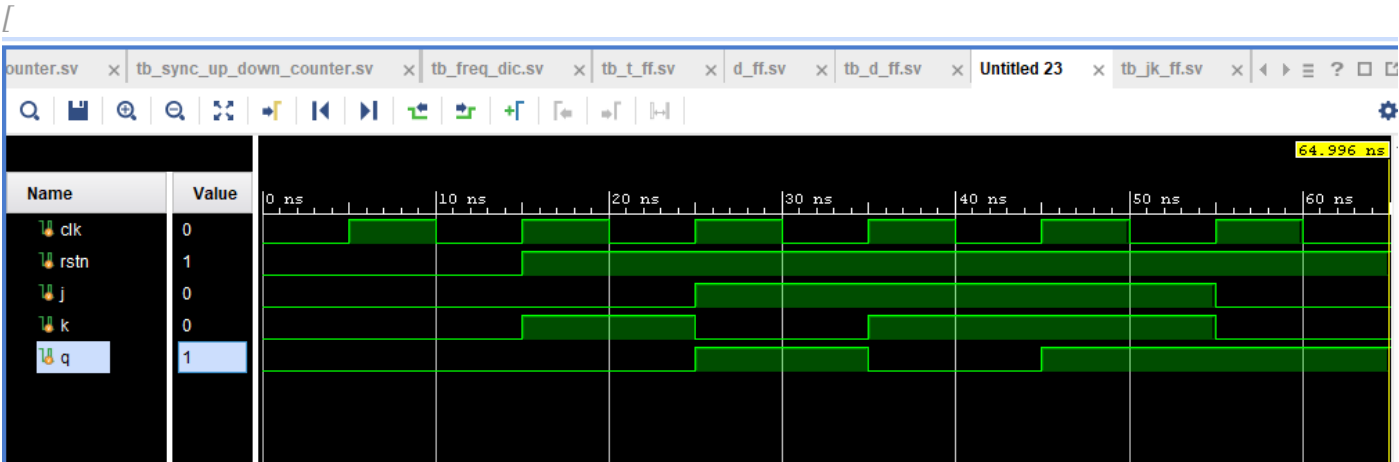
Design Code

```
tb_bcd_counter.sv  tb_siso.sv  jk_ff.sv  X
JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sources_1 > new > jk_ff.sv
1  `timescale 1ns / 1ps
2
3  module jk_ff(
4      input logic clk,
5      input logic rstn,
6      |   input logic j,
7      |   input logic k,
8      output logic q
9      );
10
11     always_ff @(posedge clk, negedge rstn) begin
12         if(!rstn) q<=1'b0;
13         |   else begin
14             case({j,k})
15             |   |   2'b00: q<=q;
16             2'b01: q<=1'b0;
17             |   |   2'b10: q<=1'b1;
18             |   |   2'b11: q<=~q;
19             endcase
20         end
21     end
22 endmodule
23
```

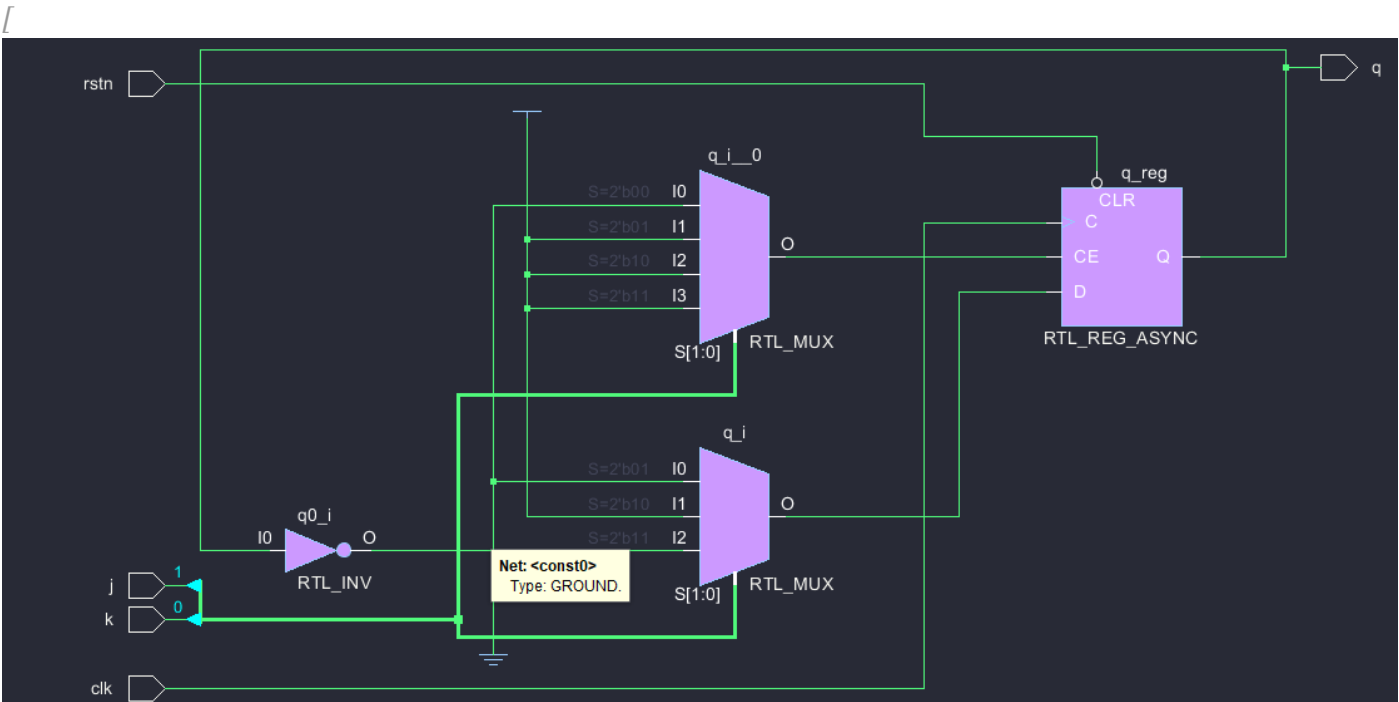
Testbench Code

```
[
tb_bcd_counter.sv tb_siso.sv tb_jk_ff.sv X
JaleedUSTP > RTL > RTLLab > week2 > week2.srsc > sim_1 > new > tb_jk_ff.sv
1  `timescale 1ns / 1ps
2
3  module tb_jk_ff();
4      logic clk;
5      | logic rstn;
6  logic j;
7      logic k;
8      | logic q;
9
10     jk_ff uut(
11         .clk(clk),
12         | .rstn(rstn),
13     .j(j),
14         | | .k(k),
15         .q(q)
16         | );
17
18     always #5 clk=~clk;
19
20     initial begin
21     clk=0;
22         rstn=0;
23         j=0;
24         k=0;
25         | #15 rstn=1;
26
27     j=0; k=1;
28         | #10 j=1; k=0;
29         | | #10 j=1; k=1;
30         #10 j=1; k=1;
31         #10 j=0; k=0;
32         | #10 $finish;
33     end
34 endmodule
35
```

Waveform



Schematic



Task 9: SR Flip-Flop

Design Code

```
tb_bcd_counter.sv  tb_siso.sv  sr_ff.sv  X
JaleedUSTP > RTL > RTLLab > week2 > week2.srsrcs > sources_1 > new > sr_ff.sv

1  timescale 1ns / 1ps
2
3  module sr_ff(
4      input logic clk,
5      input logic rstn,
6      input logic s,
7      input logic r,
8      output logic q
9  );
10
11     always_ff @(posedge clk, negedge rstn) begin
12         if(!rstn) q<=1'b0;
13     else begin
14         case({s,r})
15             2'b00: q<=q;
16             2'b01: q<=1'b0;
17             2'b10: q<=1'b1;
18             2'b11: q<=1'bx;
19         endcase
20     end
21 end
22 endmodule
23
```

Testbench Code

tb_bcd_counter.sv

tb_siso.sv

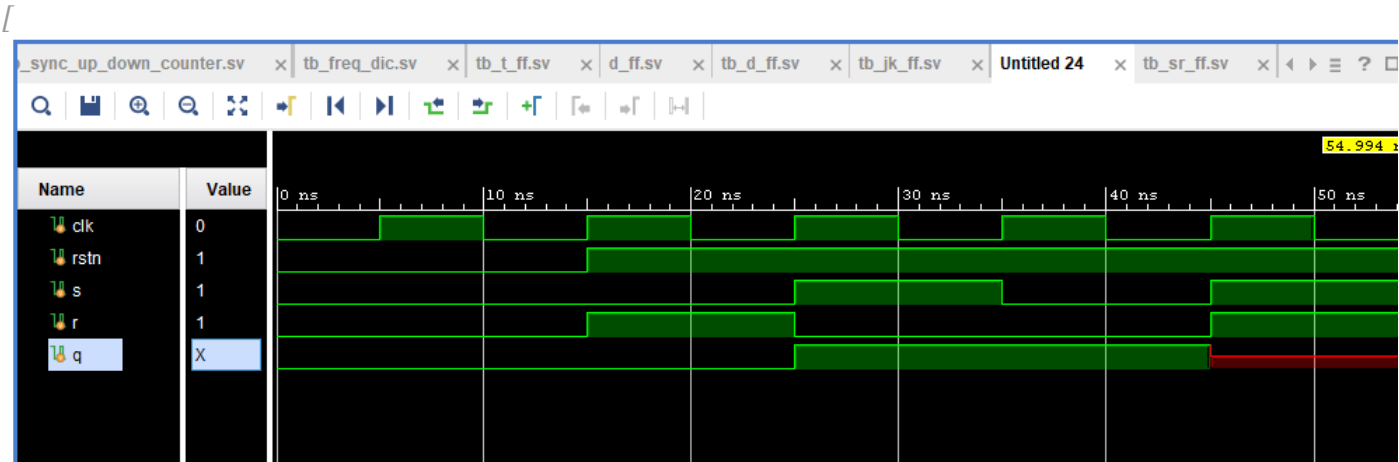
tb_sr_ff.sv



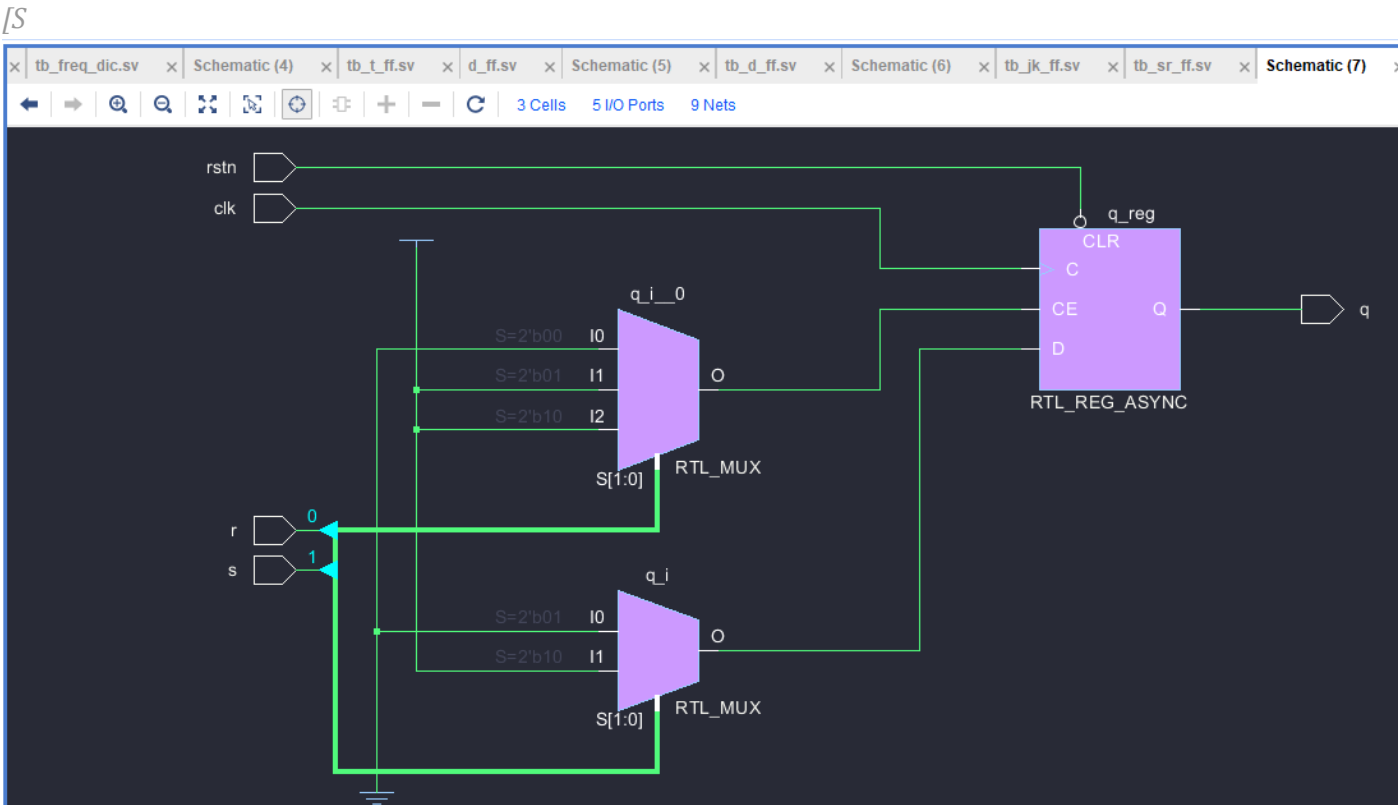
JaleedUSTP > RTL > RTLLab > week2 > week2.srsrcs > sim_1 > new > tb_sr_ff.sv

```
1  `timescale 1ns / 1ps
2
3  module tb_sr_ff();
4      logic clk;
5      logic rstn;
6      logic s;
7      logic r;
8      logic q;
9
10     sr_ff uut(
11         .clk(clk),
12         .rstn(rstn),
13         .s(s),
14         .r(r),
15         .q(q)
16     );
17
18     always #5 clk=~clk;
19
20     initial begin
21         clk=0;
22         rstn=0;
23         s=0;
24         r=0;
25         #15 rstn=1;
26
27         s=0; r=1;
28         #10 s=1; r=0;
29         #10 s=0; r=0;
30         #10 s=1; r=1;
31         #10 $finish;
32     end
33 endmodule
34
```

Waveform



Schematic



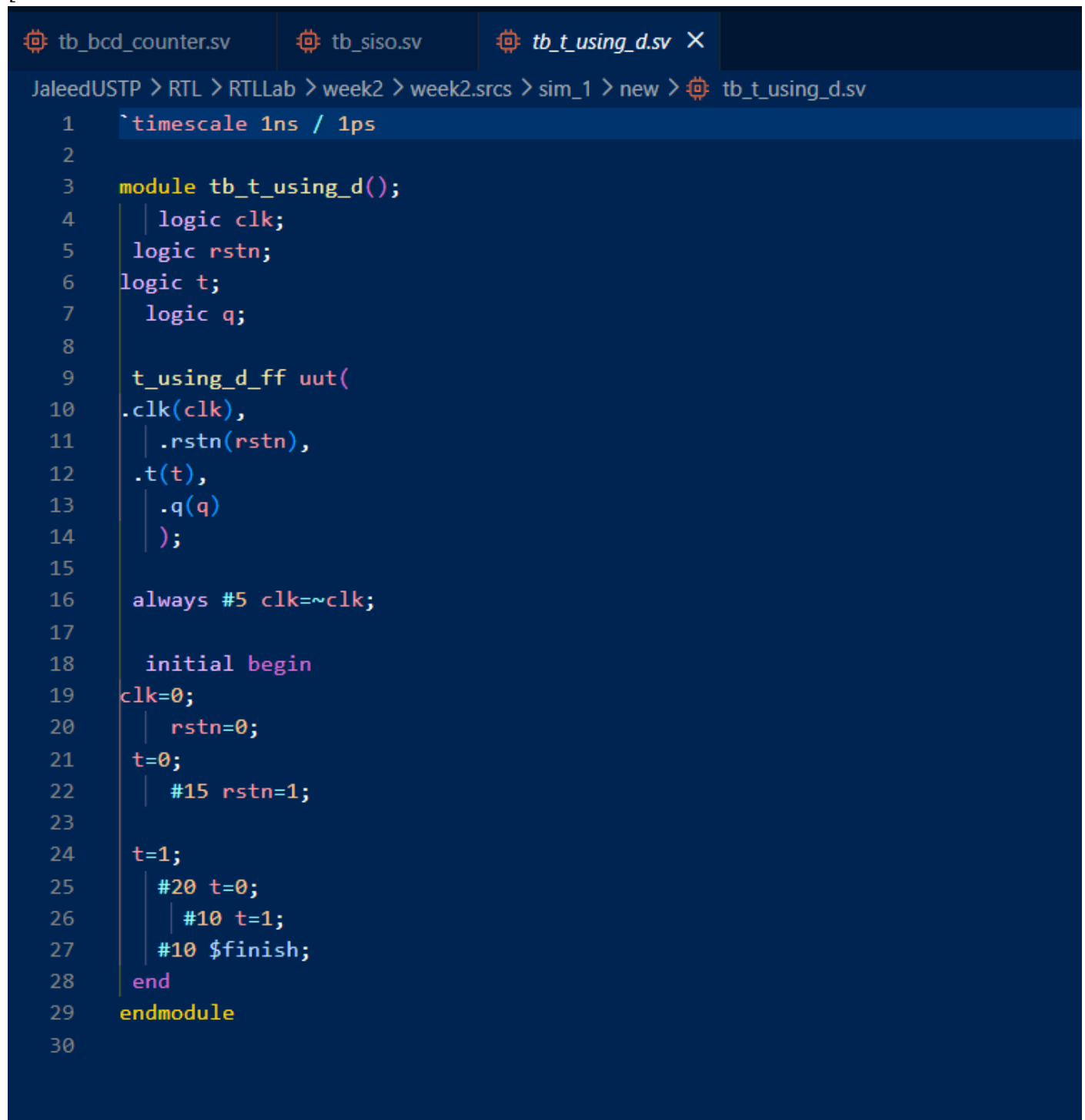
Task 10: T Flip-Flop Using D Flip-Flop

Design Code

```
tb_bcd_counter.sv  tb_siso.sv  t_using_d_ff.sv X
leedUSTP > RTL > RTLLab > week2 > week2.srscs > sources_1 > new > t_using_d_ff.sv
1  `timescale 1ns / 1ps
2
3  module t_using_d_ff(
4      input logic clk,
5      input logic rstn,
6      input logic t,
7      output logic q
8  );
9      logic d_in;
10
11     assign d_in= t ? ~q : q;
12
13     d_ff uut_d(
14         .clk(clk),
15         .rstn(rstn),
16         .d(d_in),
17         .q(q)
18     );
19 endmodule
20
```

Testbench Code

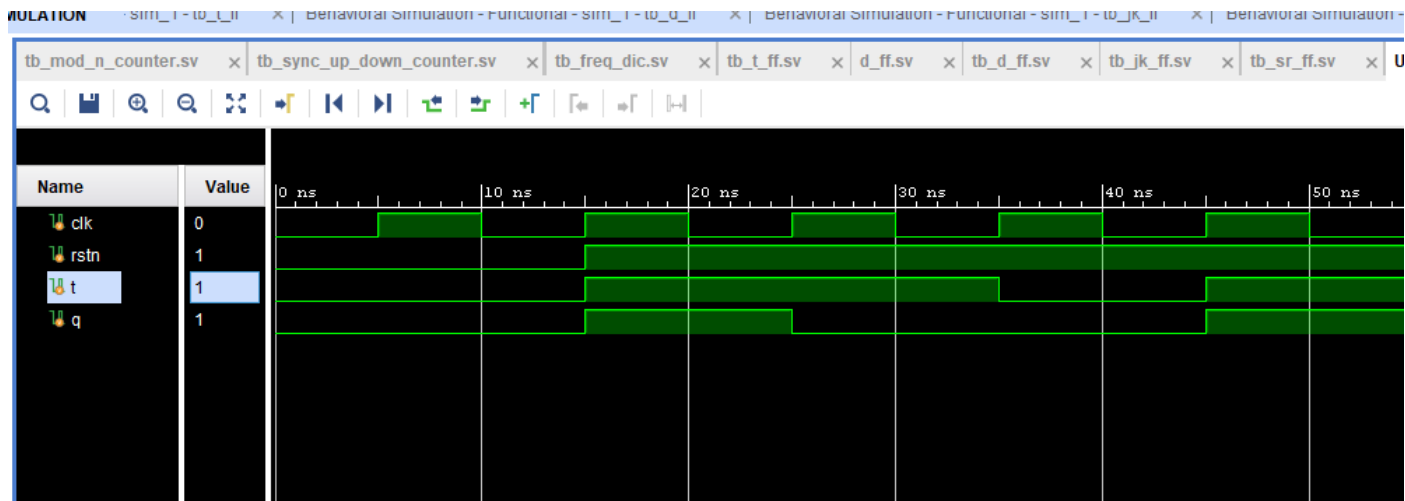
[



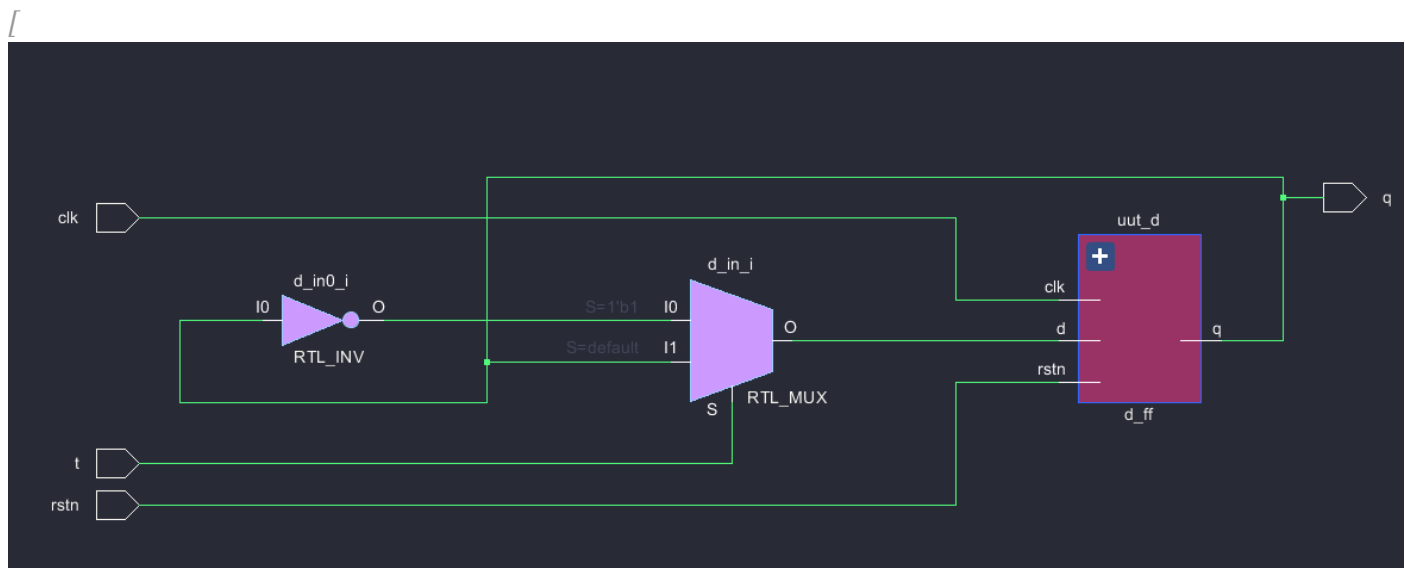
The screenshot shows a code editor with a dark theme. At the top, there is a file explorer bar with three files: `tb_bcd_counter.sv`, `tb_siso.sv`, and `tb_t_using_d.sv` (which is selected and has a close button). Below the file explorer is a breadcrumb navigation bar showing the path: `JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sim_1 > new > tb_t_using_d.sv`. The main editor area contains the following Verilog code:

```
1  `timescale 1ns / 1ps
2
3  module tb_t_using_d();
4      logic clk;
5      logic rstn;
6      logic t;
7      logic q;
8
9      t_using_d_ff uut(
10         .clk(clk),
11         .rstn(rstn),
12         .t(t),
13         .q(q)
14     );
15
16     always #5 clk=~clk;
17
18     initial begin
19         clk=0;
20         rstn=0;
21         t=0;
22         #15 rstn=1;
23
24         t=1;
25         #20 t=0;
26         #10 t=1;
27         #10 $finish;
28     end
29 endmodule
30
```

Waveform



Schematic



Task 11: SISO – Serial-In Serial-Out Shift Register

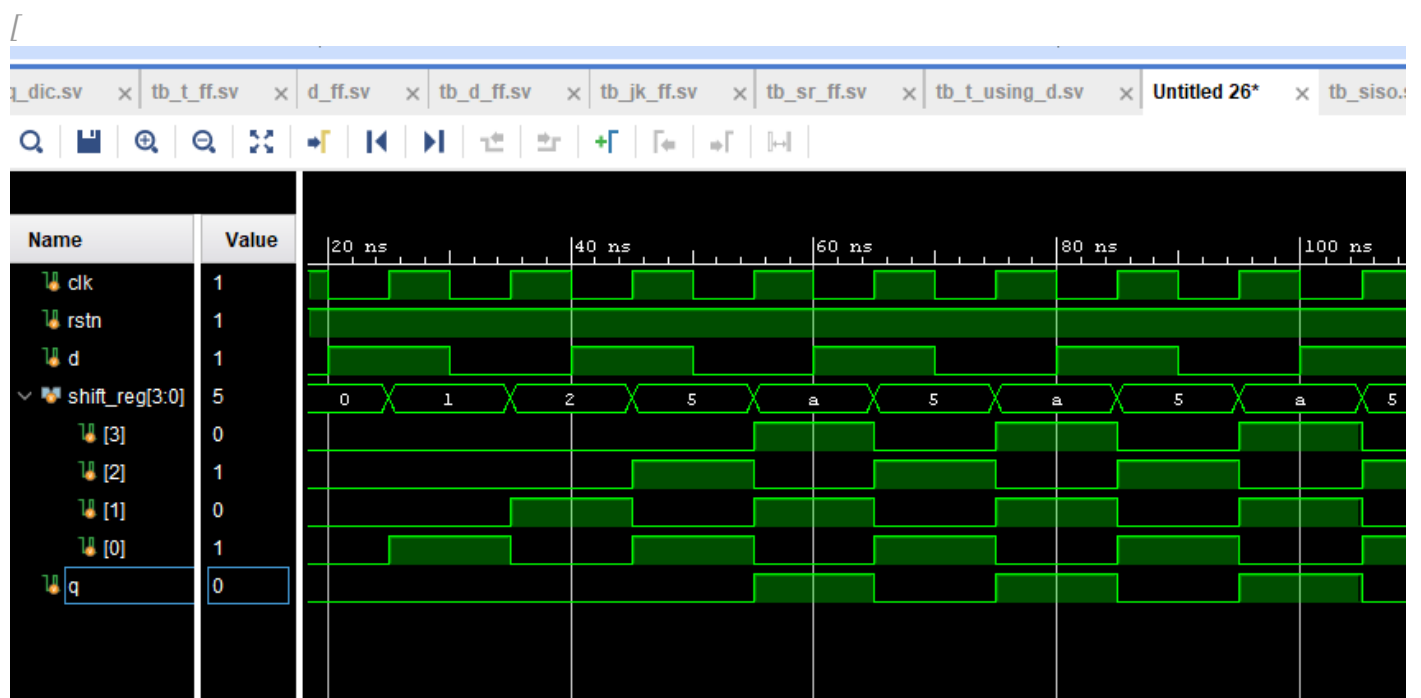
Design Code

```
tb_bcd_counter.sv  tb_siso.sv  siso.sv  X
JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sources_1 > new > siso.sv
1  `timescale 1ns / 1ps
2
3  module siso(
4      input logic clk, rstn, d,
5      output logic q
6  );
7
8  logic [3:0] shift_reg;
9
10 always_ff @(posedge clk, negedge rstn) begin
11     if(!rstn) shift_reg <= 0;
12     else shift_reg <= {shift_reg[2:0], d};
13 end
14
15 assign q = shift_reg[3];
16
17 endmodule
18
```

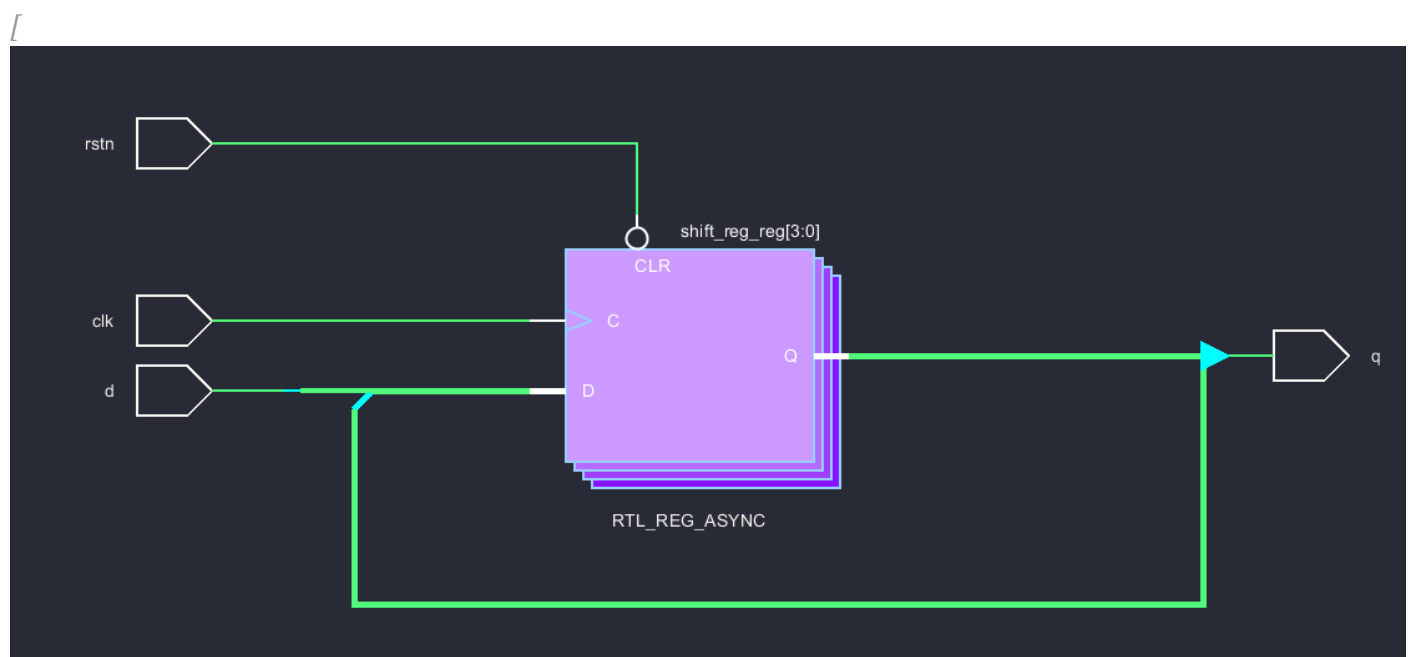
Testbench Code

```
[
tb_bcd_counter.sv tb_asiso.sv X
JaleedUSTP > RTL > RTLLab > week2 > week2.srce > sim_1 > new > tb_asiso.sv
1  `timescale 1ns / 1ps
2
3  module tb_asiso(
4      |
5      );
6      logic clk = 0;
7      logic rstn = 0;
8      logic d = 0;
9      logic q;
10
11  siso uut (
12      .clk(clk),
13      .rstn(rstn),
14      .d(d),
15      .q(q)
16  );
17
18  always #5 clk = ~clk;
19
20  initial begin
21      // Initialize signals
22      clk = 0;
23      rstn = 0;
24      d = 0;
25      #10;
26      rstn=1;
27      #10;
28      d=1;
29      #10;
30      d=0;
31      #10;
32      d=1;
33      #10;
34      d=0;
35      #10;
36      d=1;
37      #10;
38      d=0;
39      #10;
40      d=1;
```

Waveform



Schematic



Task 12: SIPO – Serial-In Parallel-Out Shift Register

Design Code

```
tb_bcd_counter.sv  sipo.sv  X
JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sources_1 > new > sipo.sv
1  `timescale 1ns / 1ps
2
3  module sipo(
4      input logic clk, rstn,
5      input logic d,
6      output logic [3:0] q
7  );
8
9  always_ff @(posedge clk, negedge rstn) begin
10      if (!rstn) q <= 4'b0;
11      else q <= {q[2:0], d};
12  end
13
14  endmodule
15
```

Testbench Code

```
tb_bcd_counter.sv  tb_sipo.sv  X
aleedUSTP > RTL > RTLLab > week2 > week2.srsc > sim_1 > new > tb_sipo.sv
1  `timescale 1ns / 1ps
2
3  module tb_sipo();
4
5      // Signal declarations
6      logic clk;
7      logic rstn;
8      logic d;
9      logic [3:0] q;
10
11     // Instantiate the Unit Under Test (UUT)
12     sipo uut (
13         .clk(clk),
14         .rstn(rstn),
15         .d(d),
16         .q(q)
17     );
18
19     // Clock generation (10ns period)
20     always #5 clk = ~clk;
21
22     // Test sequence
23     initial begin
24         clk = 0;
25         rstn = 0;
26         d = 0;
27
28         #15 rstn = 1;
29
30         #10 d = 1;
31         #10 d = 0;
32         #10 d = 1;
33         #10 d = 1;
34
35         #30 $finish;
36     end
37
38     // Monitor changes
39     initial begin
40         $monitor("Time=%0t | rstn=%b | d=%b | q=%b", $time, rstn, d, q);
41     end
42 endmodule
```

Waveform

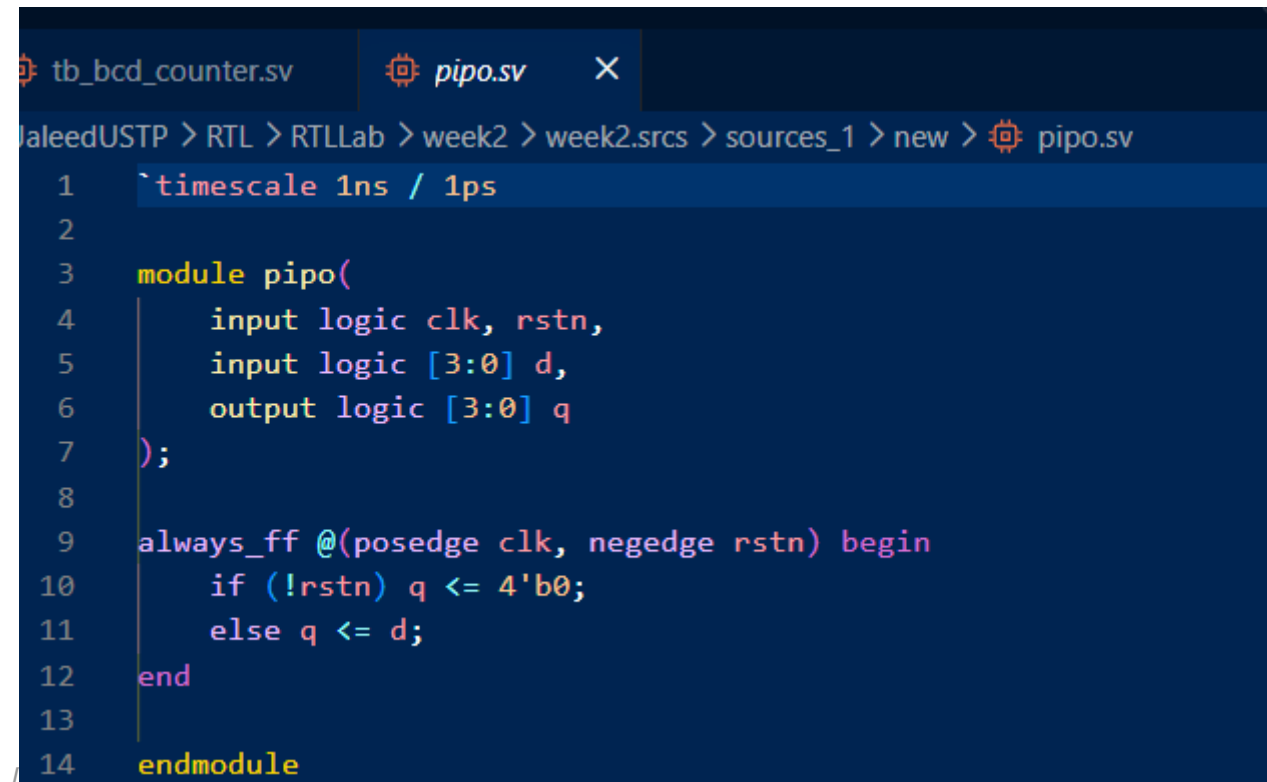
[Screenshot – paste simulation waveform here]

Schematic

[Screenshot – paste elaborated schematic here]

Task 13: PIPO – Parallel-In Parallel-Out Shift Register

Design Code



```
tb_bcd_counter.sv | pipo.sv X
jaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sources_1 > new > pipo.sv
1  `timescale 1ns / 1ps
2
3  module pipo(
4      input logic clk, rstn,
5      input logic [3:0] d,
6      output logic [3:0] q
7  );
8
9  always_ff @(posedge clk, negedge rstn) begin
10     if (!rstn) q <= 4'b0;
11     else q <= d;
12 end
13
14 endmodule
```

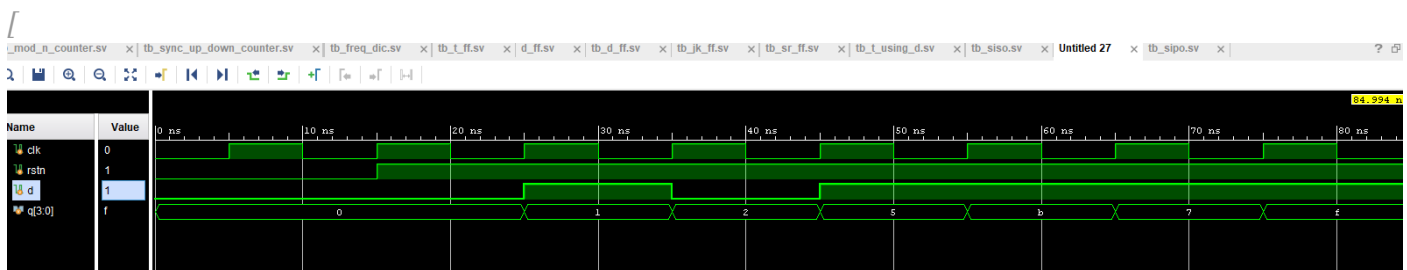
Testbench Code

```

1  `timescale 1ns / 1ps
2
3  module tb_pipo();
4
5      // Signal declarations
6      logic clk;
7      logic rstn;
8      logic [3:0] d;
9      logic [3:0] q;
10
11     // Instantiate the Unit Under Test (UUT)
12     pipo uut (
13         .clk(clk),
14         .rstn(rstn),
15         .d(d),
16         .q(q)
17     );
18
19     // Clock generation (10ns period)
20     always #5 clk = ~clk;
21
22     // Test sequence
23     initial begin
24         clk = 0;
25         rstn = 0;
26         d = 4'b0000;
27
28         #15 rstn = 1;
29
30         #10 d = 4'b1010;
31         #10 d = 4'b0101;
32         #10 d = 4'b1111;
33
34         #20 $finish;
35     end
36
37     // Monitor changes
38     initial begin
39         $monitor("Time=%0t | rstn=%b | d=%b | q=%b", $time, rstn, d, q);
40     end
41

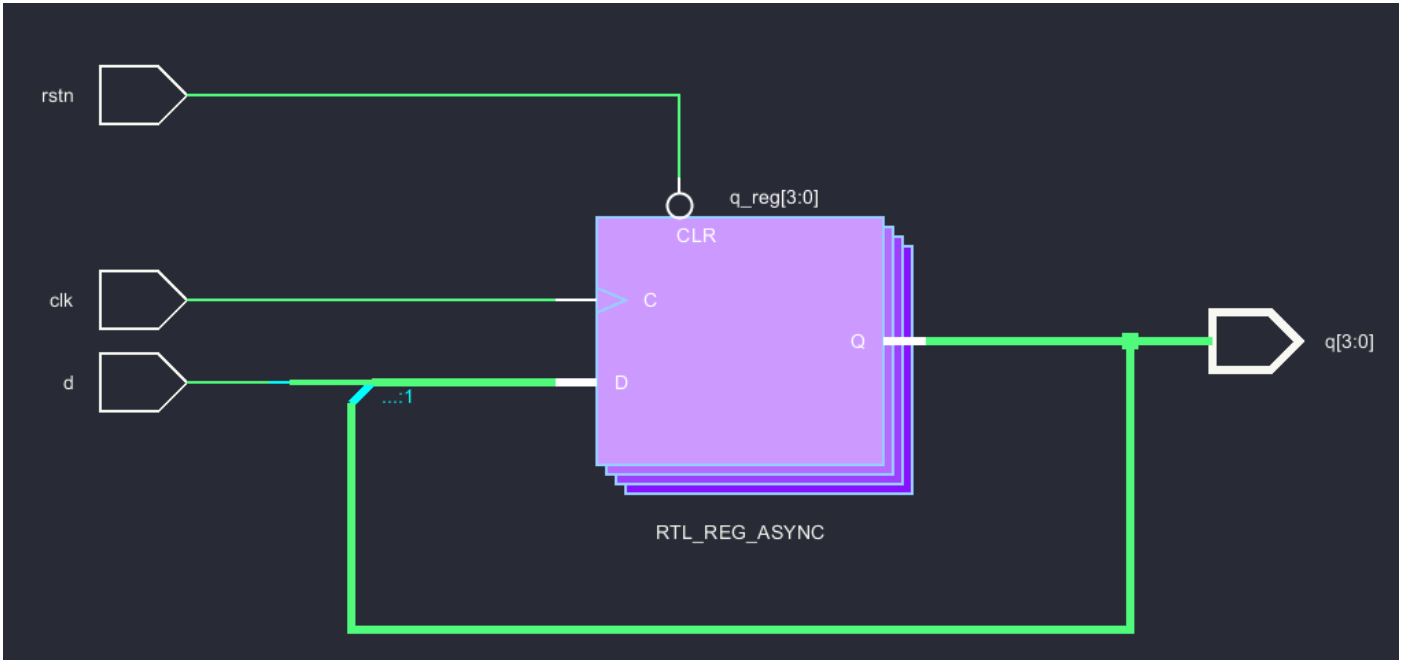
```

Waveform



Schematic

[



Task 14: PISO – Parallel-In Serial-Out Shift Register

Design Code

```
tb_bcd_counter.sv  piso.sv  X
JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sources_1 > new > piso.sv
1  `timescale 1ns / 1ps
2
3  module piso(
4      input logic clk, rstn,
5      input logic load,
6      input logic [3:0] d,
7      output logic q
8  );
9
10 logic [3:0] shift_reg;
11
12 always_ff @(posedge clk, negedge rstn) begin
13     if (!rstn) shift_reg <= 4'b0;
14     else if (load) shift_reg <= d;
15     else shift_reg <= {shift_reg[2:0], 1'b0};
16 end
17
18 assign q = shift_reg[3];
19
20 endmodule
21
```

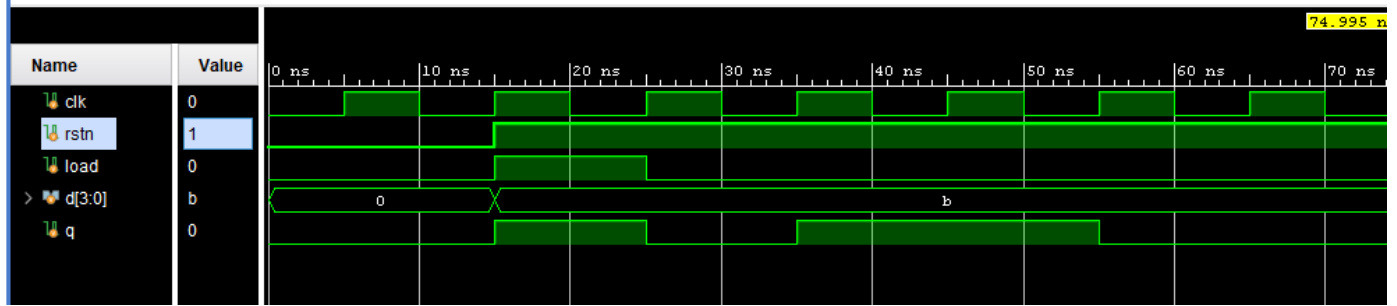
Testbench Code

tb_bcd_counter.sv tb_piso.sv X

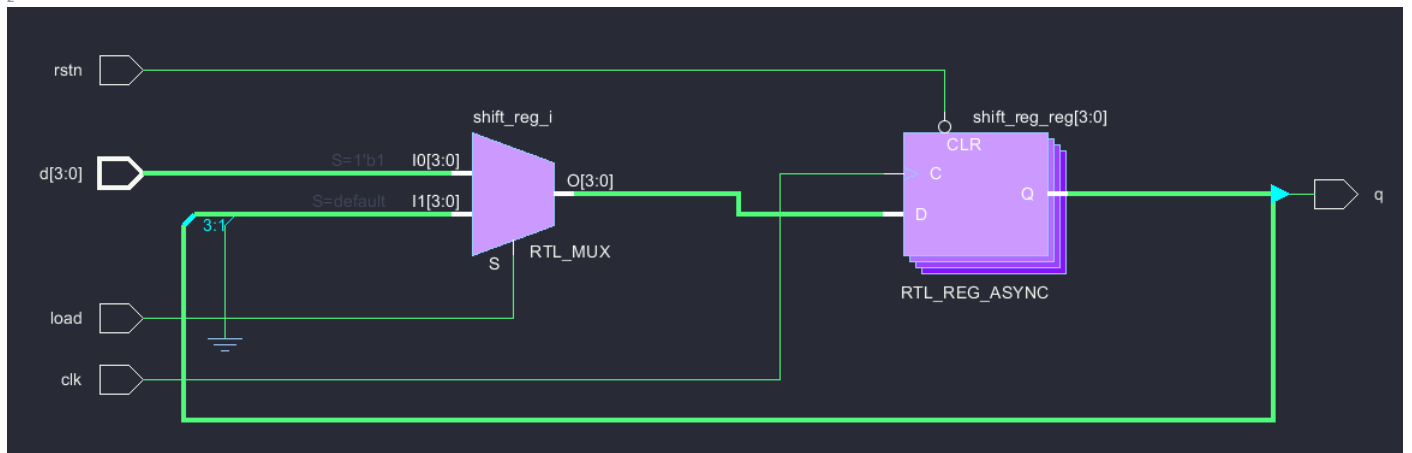
JaleedUSTP > RTL > RTLLab > week2 > week2.srsc > sim_1 > new > tb_piso.sv

```
1  `timescale 1ns / 1ps
2
3  module tb_piso();
4
5      // Signal declarations
6      logic clk;
7      logic rstn;
8      logic load;
9      logic [3:0] d;
10     logic q;
11
12     // Instantiate the Unit Under Test (UUT)
13     piso uut (
14         .clk(clk),
15         .rstn(rstn),
16         .load(load),
17         .d(d),
18         .q(q)
19     );
20
21     // Clock generation (10ns period)
22     always #5 clk = ~clk;
23
24     // Test sequence
25     initial begin
26         clk = 0;
27         rstn = 0;
28         load = 0;
29         d = 4'b0000;
30
31         #15 rstn = 1;
32
33         // Load data
34         d = 4'b1011;
35         load = 1;
36         #10 load = 0;
37
38         #50 $finish;
39     end
```

[



[



Task 15: Bidirectional Shift Register

Design Code

```
[
tb_bcd_counter.sv  bidirection_shift_reg.sv X
JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sources_1 > new > bidirection_shift_reg.sv
1  `timescale 1ns / 1ps
2
3  module bidirection_shift_reg(
4      input logic clk, rstn,
5      input logic dir,
6      input logic d,
7      output logic [3:0] q
8  );
9
10 always_ff @(posedge clk, negedge rstn) begin
11     if (!rstn) q <= 4'b0;
12     else if (dir) q <= {q[2:0], d};
13     else q <= {d, q[3:1]};
14 end
15
16 endmodule
17
```

Testbench Code

tb_bcd_counter.sv

tb_bidirectional_shift_reg.sv X

JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sim_1 > new > tb_bidirectional_shift_

```
1  `timescale 1ns / 1ps
2
3  module tb_bidirectional_shift_reg();
4
5      // Signal declarations
6      logic clk;
7      logic rstn;
8      logic dir;
9      logic d;
10     logic [3:0] q;
11
12     // Instantiate the Unit Under Test (UUT)
13     bidirection_shift_reg uut (
14         .clk(clk),
15         .rstn(rstn),
16         .dir(dir),
17         .d(d),
18         .q(q)
19     );
20
21     // Clock generation (10ns period)
22     always #5 clk = ~clk;
23
24     // Test sequence
25     initial begin
26         clk = 0;
27         rstn = 0;
28         dir = 0;
29         d = 0;
30
31         #15 rstn = 1;
32
33         // Shift left
34         dir = 1;
35         d = 1;
36         #10 d = 0;
37         #10 d = 1;
38         #10 d = 1;
39
40         #20;
```

```

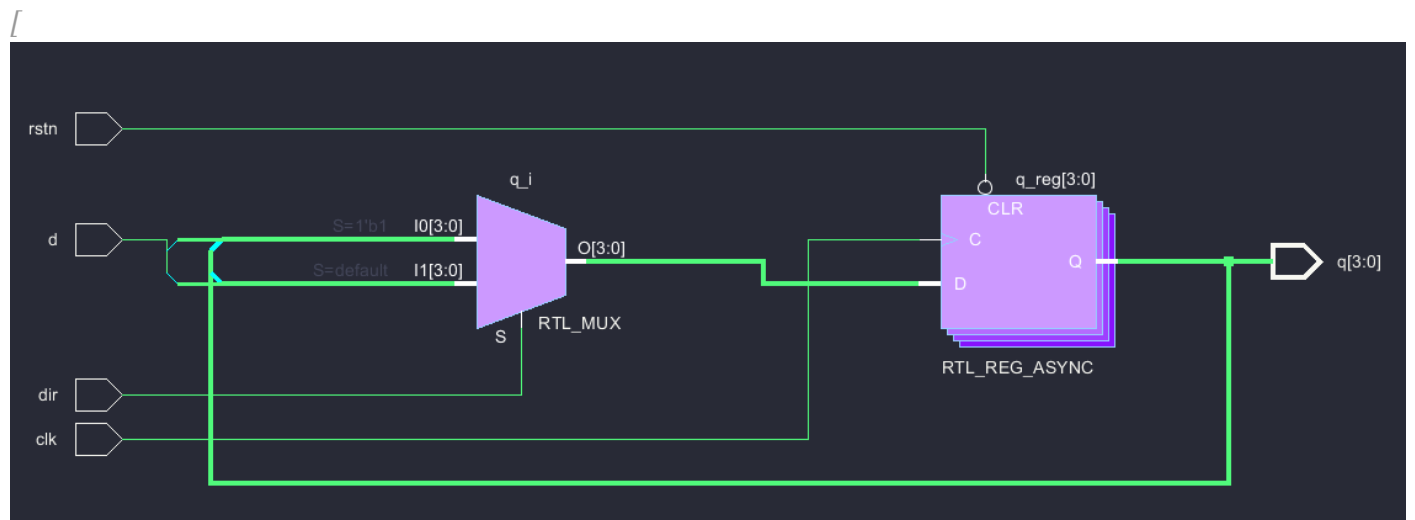
41
42     // Shift right
43     dir = 0;
44     d = 0;
45     #10 d = 1;
46     #10 d = 0;
47
48     #30 $finish;
49 end
50
51 // Monitor changes
52 initial begin
53     $monitor("Time=%0t | rstn=%b | dir=%b | d=%b | q=%b", $time, rstn, dir, d, q);
54 end
55
56 endmodule
57

```

Waveform



Schematic



Task 16: Universal Shift Register

Design Code

```
tb_bcd_counter.sv  universal_reg.sv X
JaleedUSTP > RTL > RTLLab > week2 > week2.srsc > sources_1 > new > universal_reg.sv
1  `timescale 1ns / 1ps
2
3  module universal_reg(
4      input logic clk, rstn,
5      input logic [1:0] sel,
6      input logic [3:0] p_in,
7      input logic s_left_in,
8      input logic s_right_in,
9      output logic [3:0] q
10 );
11
12 always_ff @(posedge clk, negedge rstn) begin
13     if (!rstn) q <= 4'b0;
14     else if (sel == 2'b00) q <= q;
15     else if (sel == 2'b01) q <= {s_right_in, q[3:1]};
16     else if (sel == 2'b10) q <= {q[2:0], s_left_in};
17     else if (sel == 2'b11) q <= p_in;
18 end
19
20 endmodule
21
```


Testbench Code

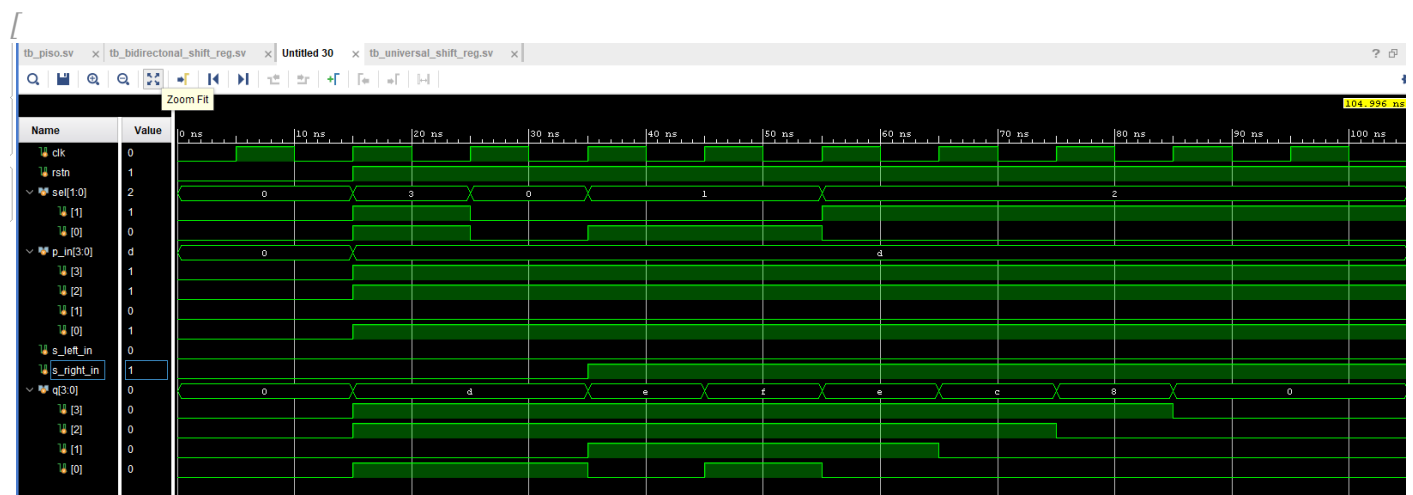
```
tb_bcd_counter.sv  tb_universal_shift_reg.sv X
JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sim_1 > new > tb_universal_shift_reg.sv
1  `timescale 1ns / 1ps
2
3  module tb_universal_shift_reg();
4
5      // Signal declarations
6      logic clk;
7      logic rstn;
8      logic [1:0] sel;
9      logic [3:0] p_in;
10     logic s_left_in;
11     logic s_right_in;
12     logic [3:0] q;
13
14     // Instantiate the Unit Under Test (UUT)
15     universal_reg uut (
16         .clk(clk),
17         .rstn(rstn),
18         .sel(sel),
19         .p_in(p_in),
20         .s_left_in(s_left_in),
21         .s_right_in(s_right_in),
22         .q(q)
23     );
24
25     // Clock generation (10ns period)
26     always #5 clk = ~clk;
27
28     // Test sequence
29     initial begin
30         clk = 0;
31         rstn = 0;
32         sel = 2'b00;
33         p_in = 4'b0000;
34         s_left_in = 0;
35         s_right_in = 0;
36
37         #15 rstn = 1;
38
39         // Parallel load
40         p_in = 4'b1101;
```

```

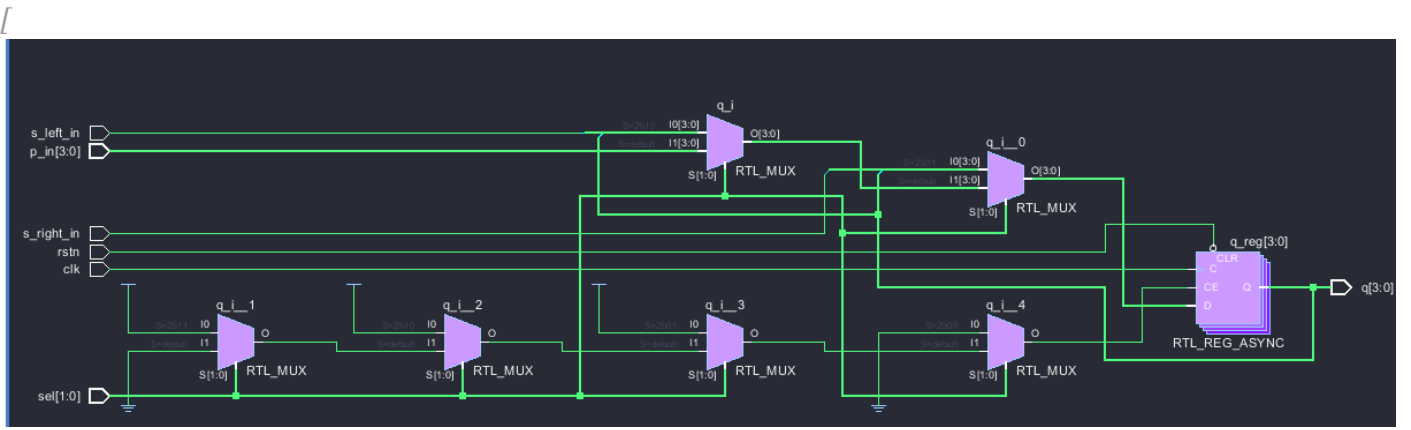
39 // Parallel load
40 p_in = 4'b1101;
41 sel = 2'b11;
42 #10;
43
44 // Hold
45 sel = 2'b00;
46 #10;
47
48 // Shift right
49 s_right_in = 1;
50 sel = 2'b01;
51 #20;
52
53 // Shift left
54 s_left_in = 0;
55 sel = 2'b10;
56 #20;
57
58 #30 $finish;
59 end
60
61 // Monitor changes
62 initial begin
63     $monitor("Time=%0t | rstn=%b | sel=%b | q=%b", $time, rstn, sel, q);
64 end
65
66 endmodule
67

```

Waveform



Schematic



Task 17: Pipelined ALU

Design Code

```
tb_bcd_counter.sv  pipelie_alu.sv X
JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sources_1 > new > pipelie_alu.sv

1  `timescale 1ns / 1ps
2
3  module pipelie_alu(
4  input logic clk,
5      input logic rstn,
6      input logic [2:0] op,
7      input logic [31:0] a,
8      input logic [31:0] b,
9  output logic [31:0] result
10 );
11
12     logic [31:0] a_reg_s1;
13     logic [31:0] b_reg_s1;
14     logic [2:0] op_reg_s1;
15
16
17     logic [31:0] alu_out_comb;
18
19     logic [31:0] result_reg_s2;
20
21     always_ff @(posedge clk, negedge rstn) begin
22         if(!rstn) begin
23             a_reg_s1 <= 32'b0;
24             b_reg_s1 <= 32'b0;
25             op_reg_s1 <= 3'b0;
26         end
27     else begin
28         a_reg_s1<=a;
29         b_reg_s1 <= b;
30         op_reg_s1<= op;
31     end
32 end
33
34     always_comb begin
35         case(op_reg_s1)
36             3'b000: alu_out_comb= a_reg_s1+b_reg_s1;
37             3'b001: alu_out_comb = a_reg_s1 - b_reg_s1;
38             3'b010: alu_out_comb=a_reg_s1 & b_reg_s1;
39             3'b011: alu_out_comb= a_reg_s1 | b_reg_s1;
40             3'b100: alu_out_comb=a_reg_s1^b_reg_s1;
```

```
39 | | 3'b011: alu_out_comb= a_reg_s1 | b_reg_s1;
40 | | 3'b100: alu_out_comb=a_reg_s1^b_reg_s1;
41 | default: alu_out_comb = 32'b0;
42 | endcase
43 | end
44 |
45 | always_ff @(posedge clk, negedge rstn) begin
46 | | if(!rstn) result_reg_s2<= 32'b0;
47 | | else
48 | | result_reg_s2<=alu_out_comb;
49 | | end
50 |
51 | assign result=result_reg_s2;
52 |
53 | endmodule
54 |
```

Testbench Code

tb_bcd_counter.sv

tb_pipeline_alu.sv X

JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sim_1 > new > tb_pipeline_alu.sv

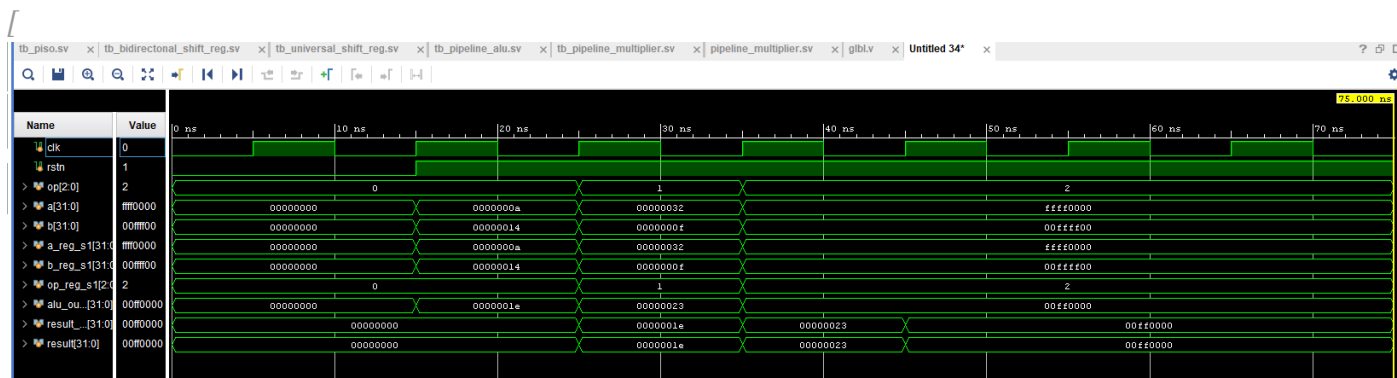
```
1  `timescale 1ns / 1ps
2
3  module tb_pipeline_alu(
4      |
5      );
6      logic clk;
7      logic rstn;
8      logic [2:0] op;
9      | logic [31:0] a;
10     logic [31:0] b;
11     logic [31:0] result;
12
13     pipelie_alu uut(
14     .clk(clk),
15     .rstn(rstn),
16     .op(op),
17     | .a(a),
18     .b(b),
19     .result(result)
20     );
21
22     always #5 clk=~clk;
23
24     initial begin
25         clk=0;
26         rstn=0;
27     op=0;
28     a=0;
29     b=0;
30     |
31     #15 rstn=1;
32
33     op=3'b000;
34     | a=32'd10;
35     b=32'd20;
36     #10;
37     |
38     op=3'b001;
39     a=32'd50;
40     | b=32'd15;
```

```

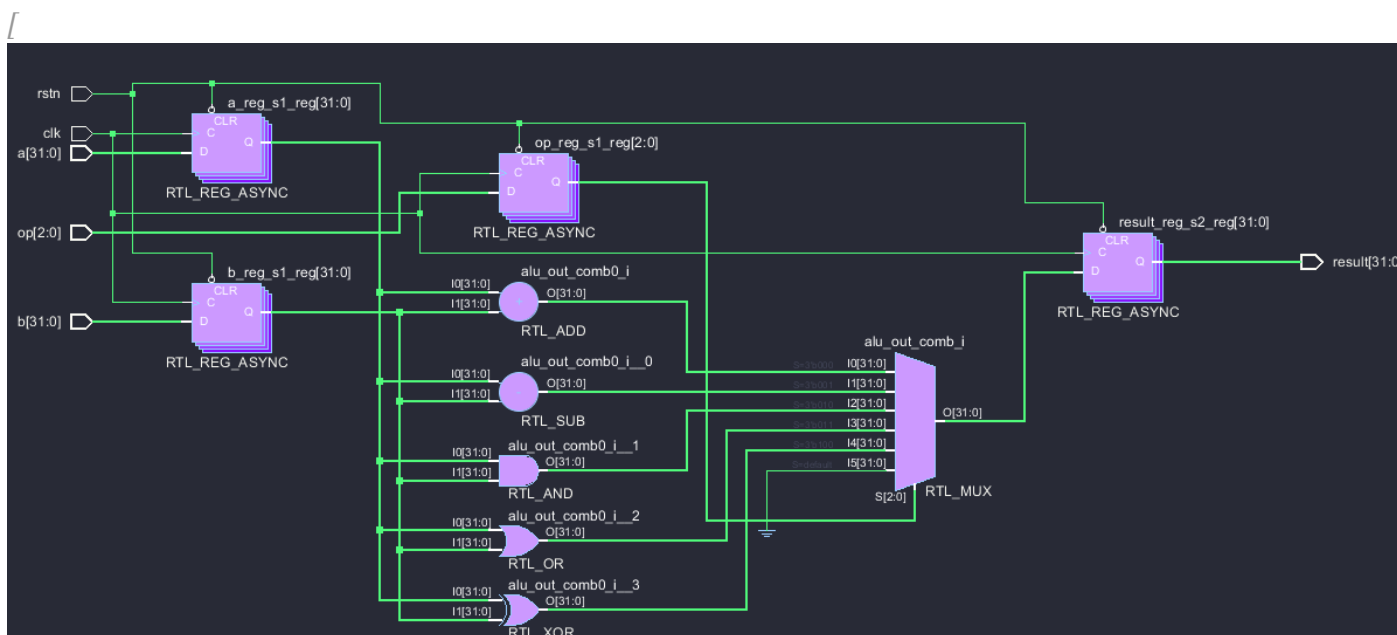
1      #10;
2
3      op=3'b010;
4          a=32'hFFFF0000;
5          b=32'h00FFFF00;
6          #10;
7
8      #30 $finish;
9      end
10
11  initial begin
12      $monitor("Time=%0t | rstn=%b | op=%b | a=%0d | b=%0d | result=%0d", $time, rstn, op, a, b, result);
13      end
14
15  endmodule

```

Waveform



Schematic



Task 18: Pipelined Multiplier

Design Code

```
tb_bcd_counter.sv  pipeline_multiplier.sv X
JaleedUSTP > RTL > RTLLab > week2 > week2.srscs > sources_1 > new > pipeline_multiplier.sv
1  `timescale 1ns / 1ps
2
3  module pipeline_multiplier(
4      input logic clk,
5      input logic rstn,
6      | input logic [31:0] a,
7      input logic [31:0] b,
8      | output logic [63:0] product
9  );
10
11     logic [31:0] a_reg_s1;
12     | logic [31:0] b_reg_s1;
13
14     logic [63:0] mult_out_comb;
15
16     logic [63:0] result_reg_s2;
17
18     always_ff @(posedge clk, negedge rstn) begin
19         | | | if(!rstn) begin
20     a_reg_s1<=32'b0;
21         | b_reg_s1<=32'b0;
22         | | | end
23         | | | else begin
24         | | | | a_reg_s1<=a;
25         | | | b_reg_s1<=b;
26         | | | end
27         | | end
28
29     assign mult_out_comb=a_reg_s1 * b_reg_s1;
30
31     | always_ff @(posedge clk, negedge rstn) begin
32     if(!rstn) result_reg_s2<=64'b0;
33     else result_reg_s2<= mult_out_comb;
34     | end
35
36     assign product= result_reg_s2;
37
38 endmodule
39
```


Testbench Code

```
tb_bcd_counter.sv  tb_pipeline_multiplier.sv X
JaleedUSTP > RTL > RTLLab > week2 > week2.srcs > sim_1 > new > tb_pip

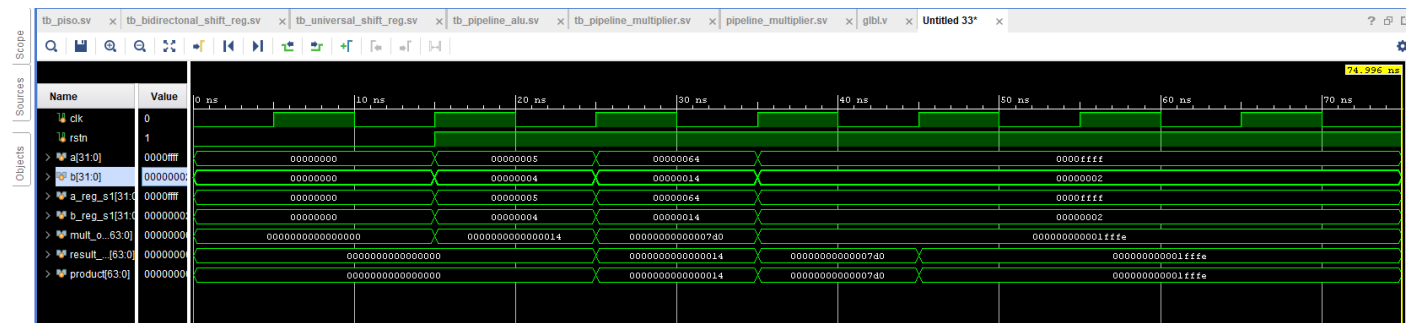
1  `timescale 1ns / 1ps
2
3  module tb_pipeline_multiplier(
4      |
5      | );
6      logic clk;
7      logic rstn;
8      logic [31:0] a;
9      | logic [31:0] b;
10     logic [63:0] product;
11
12     pipeline_multiplier uut(
13 .clk(clk),
14     .rstn(rstn),
15     .a(a),
16     .b(b),
17     .product(product)
18     );
19
20     always #5 clk=~clk;
21
22 initial begin
23     clk=0;
24     rstn=0;
25     | a=0;
26     | b=0;
27
28     #15 rstn=1;
29
30     | a=32'd5;
31     b=32'd4;
32     | #10;
33
34     a=32'd100;
35     | b=32'd20;
36     | #10;
37
38     | a=32'hFFFF;
39     b=32'h2;
40     | #10;
```

```

40     #10,
41
42     || #30 $finish;
43 end
44
45 initial begin
46     $monitor("Time=%0t | rstn=%b | a=%0d | b=%0d | product=%0d", $time, rstn, a, b, product);
47 end
48
49 endmodule
50

```

Waveform



Schematic

